

SFDB: A Context-Indexed Semantic Fact Database with Stalk Indexing and Verified Cross-Engine Equivalence

BlackSamuron0305

Abstract

Representing semantic knowledge as subject–predicate–object triples forces higher-arity and context-scoped facts through reification: one statement becomes many records, and answering a simple lookup becomes a multi-way join. We describe SFDB, a prototype database whose organising principle is instead the *context poset*: each fact is stored atomically as a section over a finite poset of contexts, and entity- and context-anchored queries are answered through precomputed indexes rather than reconstruction. The formal vocabulary is that of presheaves on a finite Alexandrov space; the sheaf condition holds vacuously there, and we claim no category-theoretic novelty. A reified triple-store baseline is co-implemented over the same canonical schema, and every benchmark query is checked, online, for canonical result equivalence across engines — a harness that caught a real semantic bug in our own engine before publication.

Across 4 scales (100, 1 000, 10 000, 100 000 facts), SFDB answers LOOKUP queries 20–32× faster, full-scan GLOBAL queries 54–99× faster, and CONTEXT queries — retrieval scoped to a context and its descendants, the query shape the design is named for — 2.97–21× faster than the baseline. A storage-layer control — the same baseline with its SQLite tables replaced by in-process dict indexes — shows that most of the LOOKUP, GLOBAL, and CONTEXT advantage (8.1–16×, 19–39×, and 1.59–9.0×) survives with the storage-layer difference removed and is therefore attributable to the design itself. The temporal-range advantages do not survive the control at all — SFDB is at parity or measurably slower than the control on bounded temporal ranges at most scales — and insert throughput is slower than the baseline at the smallest scale measured and faster at every larger one; both are reported as negative or mixed results rather than smoothed into a single headline. The cost that funds the advantages that do hold is memory: 1.21–2.71× more resident memory per fact than the baseline. Against three real, disk-backed production stores accessed over SPARQL/HTTP — Apache Jena TDB2, OpenLink Virtuoso, and Ontotext GraphDB, independently engineered by three different organisations — the pattern holds up on LOOKUP, GLOBAL, and CONTEXT — SFDB remains faster than all three at every scale, despite their latencies including an HTTP round trip SFDB’s in-process calls do not pay — but reverses on TEMPORAL from 10 000 facts onward against all three stores, where a mature production optimiser overtakes SFDB’s Python-level candidate filtering. We report this crossover as measured rather than smoothing it into either a clean win or a hedge; that it appears independently against three unrelated optimisers is evidence it is a property of mature query engines generally rather than one vendor’s implementation. Two further comparisons — 5,337 genuine Wikidata facts we did not construct, and LUBM, the standard RDF benchmark workload, at four scales up to 97,289 facts — extend the finding rather than merely repeat it: LOOKUP, GLOBAL, and CONTEXT reproduce cleanly on both, while TEMPORAL lands in a different place on each (beating both in-house baselines on Wikidata, roughly at parity with them on LUBM), direct evidence that only TEMPORAL among the four query classes is genuinely sensitive to a dataset’s actual shape. Building the LUBM comparison surfaced the evaluation’s most consequential self-correction: a performance bug in SFDB’s own CONTEXT query resolution, present for the entire evaluation and exposed only by a topology shape (many large, mutually unrelated top-level contexts) none of this paper’s other datasets have, which required re-running and re-verifying every CONTEXT number this paper reports and retracting two earlier claims built on the unfixed

numbers. Four external reference points (rdflib, Jena TDB2, Virtuoso, and GraphDB, all five query classes, result counts verified against all four), two real-world/standard-workload comparisons, and ten documented rounds of self-correction — including the CONTEXT bug above and a cross-engine SPARQL semantics divergence found while building the Virtuoso comparison — complete an evaluation built to be checked rather than believed. All results regenerate from the released artifact with a single command.

1 Introduction

Knowledge graphs have become the default representation for semantic information on the web and in enterprise systems. Their foundations are simple: every fact is a triple (s, p, o) that asserts a binary relationship between a subject, a predicate, and an object [10, 13]. The model is uniform, transparent, and has an enormous accumulated tooling base. It is also a poor fit for large classes of real data.

Two mismatches recur across domains. First, many facts are not binary. A drug interaction record carries a drug, a target, a dosage, a route, a study population, and a confidence interval, all in one statement. A financial transaction pairs a buyer, a seller, an instrument, a price, a currency, a venue, and a timestamp. Encoding such facts as triples requires *reification*: an auxiliary event node is minted, each attribute becomes its own triple pointing at that node, and reconstructing the original fact costs one join per attribute. Second, the same fact often holds in some contexts and not in others. A biological interaction that is valid *in vitro* may fail *in vivo*; a corporate policy that applies in the European subsidiary may not apply in North America. Context is not first-class in the triple model. It is added either through separate named graphs [6], through further reification, or through ad-hoc conventions in the predicate URI.

Both problems have well-known workarounds, and mature triple stores handle them competently at scale. The question we pose in this paper is whether there is anything to gain from a database whose primary organising principle is not the triple but the *context*. Concretely, we ask whether one can build a storage engine that (i) stores each fact as an atomic record, without reification, and (ii) indexes those records by the context poset in a way that answers entity-centric queries via a direct index lookup while remaining verifiably equivalent to a conventional RDF-style store on the queries that both engines can answer.

The mathematical vocabulary that fits this question naturally is that of presheaves on a poset of contexts. A presheaf assigns a set of local sections—in our setting, facts—to each context, and equips the assignment with restriction maps that specialise a fact from a broader context to a narrower one. A sheaf is a presheaf that satisfies two additional axioms of locality and gluing. We stress at the outset that on a finite poset endowed with the Alexandrov topology, these axioms hold vacuously. The system we describe is therefore a *presheaf-based* database in the strict technical sense; the sheaf terminology is retained because the operational vocabulary of restriction, stalks, and global sections is what our implementation actually exposes, not because we claim a non-trivial sheaf-theoretic theorem.

Contributions

The contributions of this paper are, in order:

1. A concise formal model of a context-indexed semantic fact store, expressed in the language of presheaves on a finite poset (Section 6). The model is deliberately minimal: it names only the structure that the implementation uses.
2. A prototype implementation, the *Semantic Fact Database* (SFDB), that realises the model in roughly 16 500 lines of Python (Section 10). Two storage engines are provided over a shared canonical schema: a reference triple store with SPO/POS/OSP indexes and

reification for n -ary facts, and a sheaf-inspired engine with a stalk index, a restriction cache, and an on-demand global-section constructor.

3. A canonical intermediate representation together with a bidirectional mapping to and from each engine’s native format, used to establish semantic equivalence at every insert and every query (Section 7). Cross-engine equivalence is enforced by the test suite (416 passing tests) and re-checked on every benchmark run.
4. A benchmark evaluation across five query classes (LOOKUP, GLOBAL, CONTEXT, bounded TEMPORAL, and unbounded TEMPORAL) and insert throughput, at 4 scales (100, 1 000, 10 000, 100 000 facts; Section 11), with every query at every scale checked for three-way cross-engine result equivalence as part of the same run. The evaluation includes a *storage-layer control*: a variant of the KG baseline with identical reification and query logic whose SQLite tables are replaced by in-process dict indexes, so that the comparison separates what context-indexed storage buys from what merely avoiding SQLite round trips buys (Section 11.4), plus four external reference points: rdfib, a real independent Python library (Section 11.5), and three real production RDF stores accessed over SPARQL/HTTP, Apache Jena TDB2 (Section 11.6), OpenLink Virtuoso (Section 11.7), and Ontotext GraphDB (Section 11.8) — with result counts verified against all four on every class at every scale. The sheaf-inspired engine is 20–32 $\times$ faster than the KG baseline on LOOKUP queries (8.1–16 $\times$ against the control), 54–99 $\times$ faster on GLOBAL scans (19–39 $\times$ against the control), and 2.97–21 $\times$ faster on CONTEXT queries — the query shape the design is named for — (1.59–9.0 $\times$ against the control); against all three real production stores, SFDB remains faster on LOOKUP, GLOBAL, and CONTEXT at every scale but is overtaken on TEMPORAL from 10^4 facts onward, a genuine crossover, reproduced independently against three unrelated optimisers, that we report as measured rather than as either a clean win or a hedge. The temporal-range advantages over the SQLite baseline do not survive the KG-mem control at all and are reported as a negative result, and insert throughput is slower than the write-through baseline at the smallest scale measured and faster at every larger one. The tradeoff we report is memory: SFDB’s in-memory indexing structures use 1.21–2.71 $\times$ more resident memory per fact than the KG baseline’s reified-triple storage. We also report that tightening the cross-engine verification harness during this evaluation caught a real semantic bug in the sheaf engine’s LOOKUP path (Section 7), which we take as evidence for the verification methodology, not just for the numbers. Two further comparisons, a real-world case study (Section 11.9, 5,337 genuine Wikidata facts) and the LUBM standard benchmark workload (Section 11.10, up to 97,289 facts), confirm the LOOKUP/GLOBAL/CONTEXT advantage on both while leaving TEMPORAL in a different place on each, and building the LUBM comparison surfaced a genuine CONTEXT-resolution performance bug (Section 12.5) that had been present throughout the entire evaluation, requiring every CONTEXT number in this paper to be re-measured after the fix.
5. A discussion of the design’s realistic scope, including the honest observation that the sheaf condition is vacuous in our setting and therefore not the source of any of the performance differences we observe (Section 12). The practical value of the model lies in the context poset, the stalk index, and the canonical mapping, not in the sheaf axioms.

What This Paper Is Not

We do not claim that a sheaf-inspired database replaces production RDF stores such as Apache Jena, GraphDB, or Virtuoso. Our KG baseline is a research prototype implemented by the same authors as the SFDB engine, and we call attention to this limitation throughout: the LOOKUP, GLOBAL, and CONTEXT speedups against our own baselines are best interpreted

as measurements of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering quality, not as claims about the state of the art in RDF engines. We do report direct comparisons against three real production stores, Apache Jena TDB2 (Section 11.6), OpenLink Virtuoso (Section 11.7), and Ontotext GraphDB (Section 11.8) — SFDB is faster than all three on LOOKUP, GLOBAL, and CONTEXT, and slower than all three on TEMPORAL from 10^4 facts onward — and against LUBM (Section 11.10), the standard benchmark workload for RDF stores, at scales up to 97,289 facts, but not yet against BSBM or WatDiv, or LUBM at the much larger scales published LUBM results use; extending further in that direction remains future work. We do not claim novelty in category theory. The formal machinery we use is standard.

Paper Organisation

Section 2 places the work relative to existing knowledge graph and category-theoretic database literature. Section 4 recalls the finite mathematical background we rely on. Section 5 states the storage and equivalence problems formally. Section 6 presents the formal model, together with the definitions and lemmas that back it. Section 9 describes the system architecture, and Section 10 the implementation. Section 11 reports the benchmark results, organised by query class. Section 12 interprets the results, and Section 13 states the limitations. Section 14 outlines future work, and Section 15 concludes.

2 Related Work

Our work sits at the intersection of three research traditions. This section places it relative to each of them and states, plainly, what we do and do not adopt from the mathematical vocabulary we borrow.

Knowledge graphs and semantic stores. The RDF triple model [13] and its query language SPARQL [10] form the backbone of the modern semantic web. Production triple stores—Apache Jena, Virtuoso, GraphDB, Blazegraph—have evolved highly tuned index structures (SPO, POS, OSP, and permutations), custom storage formats, and cost-based query optimisers for the reified triple representation. Named graphs [6] extend the triple with a fourth graph coordinate and are the standard mechanism for scoping. Property graphs [3], as implemented in Neo4j and Amazon Neptune, decorate edges with attribute maps, which handles some higher-arity cases gracefully but reintroduces the reification problem for facts of arity above two or three (Section 2 returns to this comparison in more depth below).

The reification cost of representing n -ary facts in triples is well documented [4, 15]. It is $O(k)$ triples and $O(k)$ joins per fact of arity k . Our KG baseline implements the standard reified approach with SPO/POS/OSP indexes and dictionary encoding. We do not claim this baseline is state of the art; it is a controlled reference point.

Statement-level annotation and n -ary mitigation. The reification tax has motivated a family of mitigations inside the triple model itself. RDF-star [11, 12] allows a triple to appear as the subject or object of another triple, so statement-level metadata (provenance, confidence, temporal scope) no longer needs a minted event node; the singleton-property encoding [14] achieves a similar effect by minting a unique predicate per statement. Both are genuine improvements over classical reification for the *annotation* use case, and both are a different bet than ours. RDF-star keeps the triple as the storage unit and adds a nesting mechanism on top: a fact of arity k is still k triples plus quoted-triple annotations, an entity-anchored query still reassembles the fact through joins, and context remains an annotation on statements rather than an index over them. SFDB inverts that bet: the atomic storage unit is the whole n -ary fact,

and the context poset is the primary index, so entity-anchored retrieval is a stalk lookup rather than a join, at the memory cost quantified in Section 11.11. Whether RDF-star’s engine-level implementations (which are maturing quickly) close the retrieval gap in practice is an empirical question our evaluation does not answer; the architectural distinction — annotate statements versus index by context — is the point this paper explores.

Labeled property graphs. Labeled property graphs (LPGs) [3], as implemented by Neo4j, Amazon Neptune, and TigerGraph, are the other obvious answer to “why not just use a system that already handles n -ary facts.” An LPG attaches a key-value property map directly to each node and edge, so a binary relation with metadata (an `employed_at` edge carrying a `since` property) needs no reification at all. This genuinely dissolves the reification problem for facts whose natural arity is two: subject and object are the edge’s endpoints, and everything else is a property on that one edge, retrieved with the edge in a single lookup. It does not dissolve it for facts of higher arity, which is the case this paper is about: a property map is attached to one edge between two nodes, so a fact with three or more essential participants (a grant awarded to a recipient by a funder for a project) still needs either a synthetic intermediate node with several edges radiating from it — the same reification this paper’s KG baseline performs, just drawn as a graph instead of a triple store — or an awkward encoding as edge properties on one arbitrarily chosen pair of participants, which asymmetrically privileges two of the n participants over the rest. Context is a second, independent gap: LPG query languages (Cypher, Gremlin) have no native notion of a context poset to index by. Scoping in practice is done either with property filters evaluated per node/edge at query time (linear in the matched set, not a direct lookup) or with separate physical databases per tenant, neither of which offers the direct `context:<c>` open-set lookup or the restriction-map structure this paper’s context poset provides (Section 6.3). The comparison is therefore not “sheaf-indexed store versus property graph” in the abstract; it is that LPGs solve a real subset of the reification problem this paper opens with — binary facts with metadata — cleanly and natively, while leaving both higher-arity reification and hierarchical context indexing as open problems on top of the same substrate this paper’s KG baseline already represents. We do not benchmark against Neo4j or Neptune directly: our KG baseline’s reified-triple SPO/POS/OSP design is architecturally closer to the production RDF stores we do compare against (Section 11.6–Section 11.8) than to an LPG’s native edge-property storage, and a fair LPG comparison would require a fourth adapter and a schema translation this paper does not attempt; we name it here as the most natural fourth-store extension (Section 14.3) rather than claim a result we did not measure.

Categorical and topological database models. The categorical database literature [19] recasts schemas as categories and instances as functors. Sheaf structures have been used for distributed sensor fusion [17] and for reasoning about locally consistent but globally inconsistent data collections, where obstructions to global sections characterise contextuality [1]. Simplicial complexes and cellular sheaves have been proposed as data models for higher-order relationships [7]. Formal Concept Analysis [8] organises data around a lattice of concepts that shares surface structural features with our context poset, but does not include the restriction maps or stalks that our design uses operationally. Incidence algebras over locally finite posets [18] are, in our own survey of candidate formalisms, the nearest mathematical neighbour to the context-poset design: they support Möbius-inversion style aggregation over exactly the poset structure we index by, and we regard an incidence-algebra query layer as the most promising follow-up formalism (Section 14).

We borrow terminology and structural intuition from this literature, but we are not proving a theorem about sheaves. As we make explicit throughout, the sheaf condition is vacuous on the finite Alexandrov topology that fits our use case, so anything the model buys us empirically is buying it because of the presheaf structure and the choice of indexing, not because of the sheaf

axioms.

Context and provenance in semantic data. Contexts, named graphs, and provenance annotations have received sustained attention in the semantic web community [6]. Approaches range from explicit named graphs, to context-carrying quads, to reified provenance ontologies such as PROV-O. Our contribution is neither a new context ontology nor a new provenance model; we adopt a straightforward dot-separated hierarchy of contexts as our first-class organising principle and show what a database built around it looks like when the mathematical structure is taken seriously.

Positioning. To the best of our knowledge, no prior system provides all of the following together: a context-indexed semantic fact storage engine with stalk-based LOOKUP, a co-implemented reified triple-store baseline, a canonical intermediate representation with a constructive semantic-equivalence proof between the two that a runtime harness checks on every insert and query, and a benchmark evaluation that reports both wins and losses across a LOOKUP/GLOBAL/CONTEXT/TEMPORAL taxonomy. The individual ingredients are all standard, in some cases decades old. What this paper contributes is a coherent design that combines them and an honest evaluation of what the combination is and is not worth.

3 Motivation

We motivate the design by working through a concrete example that recurs, in various forms, across our target domains. Consider the following statement, recorded in a biomedical knowledge base:

Aspirin, at a dose of 100 mg per day, taken orally by adult subjects, inhibits COX-1, with a confidence of 0.92, according to a randomised trial published in 2019, in the clinical-adult context.

The statement is a single fact about a single relationship. Its arguments are the compound (*aspirin*), the dose (100 mg/day), the route (*oral*), the population (*adult*), the target (*COX-1*), the outcome (*inhibition*), the confidence (0.92), the source (*a randomised trial published in 2019*), and the context of validity (*clinical, adult*). There are two things to notice.

3.1 The Cost of Reifying n -ary Facts

Recorded as RDF triples, the statement above cannot fit into a single (s, p, o) record. The standard workaround is reification: a fresh event node e is minted, the fact is asserted as $(e, \text{rdf:type}, \text{Fact})$, and each argument becomes its own triple pointing at e . For a fact with k arguments, one obtains $O(k)$ triples plus the event-type triple; the inverse operation, reconstructing the original fact from the triples, requires k joins on the event node. On our example this yields roughly ten triples in place of one statement, and any query that touches multiple arguments—“all compounds that inhibit COX-1 at oral doses above 50 mg”—must join across all of them.

Named graphs [6] and property graphs [3] soften this problem in specific ways: named graphs attach a fourth coordinate (the graph name) to each triple, and property graphs decorate edges with attribute maps. Both mechanisms improve on pure RDF for particular workloads, but neither of them removes the underlying atomicity mismatch: the storage unit is smaller than the semantic unit, and the cost of reassembly is paid on every query.

3.2 Contexts Are Not First-Class

The example also contains a *context*: the statement is asserted *in the clinical-adult context*. The same clinical fact, restricted to a paediatric population, may fail. In practice, contexts are usually encoded either through named graphs (one graph per context), through further reification (each fact carries a `sfdb:context` triple), or through convention in the predicate URI. All three approaches work, but none of them makes context part of the database’s core index structure. Queries that scope naturally by context—“all facts that hold in the *clinical.adult* context”—end up as filters or joins over auxiliary data rather than as primary-index lookups.

3.3 What a Context-Indexed Store Would Buy

Suppose that, instead of triples, the primary storage unit were the full fact—the tuple of all arguments plus the context—and that the primary index were keyed by context. Two consequences follow immediately. A lookup that resolves a specific entity within a specific context becomes a single hash probe into the stalk at that context, returning the complete fact without joins. A query that scopes to a sub-tree of the context poset touches exactly the sections attached to that sub-tree, without scanning the whole store — directly answering the “all facts that hold in the *clinical.adult* context” question above. Neither observation is deep; both are essentially bookkeeping. The question is whether the bookkeeping pays for itself in practice, whether it is a property of the design rather than an artefact of the baseline it is compared against, and what it costs on queries it does not help — range scans and, structurally, resident memory.

The remainder of the paper answers those questions concretely. Section 6 names the structure precisely and identifies the minimal presheaf machinery that our implementation actually uses. Sections 9 and 10 describe the storage engine that realises it. Section 11 measures the benefit on LOOKUP queries and, directly answering the “all facts that hold in the *clinical.adult* context” question posed above, on CONTEXT queries — plus a storage-layer control (Section 11.4) that checks whether either benefit is a property of the design or merely of avoiding a database round trip — on identical workloads verified for semantic equivalence against a conventional triple-store baseline.

3.4 Research Gap

Sheaf theory has been applied to distributed computing, sensor fusion [17], and categorical database theory [19]. Formal Concept Analysis [8] uses similar lattice structures for concept hierarchies but does not include restriction maps or a notion of stalk. To the best of our knowledge, no prior system combines a context-indexed storage layer with a stalk-based query path and a runtime-verified equivalence argument against a triple-store baseline on the same core schema.

4 Background

This section names, at a high level, the mathematical ideas the formal model in Section 6 specialises to our setting; it does not restate them as separate definitions, to avoid maintaining two descriptions of the same structure. A poset $(\mathcal{C}, \leq)$ of contexts carries the *Alexandrov topology*: a subset is open iff it is downward-closed under $\leq$, so that every point has a smallest open neighbourhood. A *presheaf* on $\mathcal{C}$ assigns a set of local sections to each context together with restriction maps that specialise a section from a broader context to a narrower one; a *sheaf* is a presheaf additionally satisfying locality and gluing. On the finite, Alexandrov-topologised poset we use, both axioms hold vacuously, which is why Section 6.4 treats this as a presheaf database that borrows sheaf-theoretic vocabulary rather than as a claim of a non-trivial sheaf-theoretic

result. Section 6 gives the precise, paper-specific versions of context, presheaf, restriction, stalk, and global section that the implementation and the equivalence proof (Section 7) actually use.

5 Problem Statement

We formalise the storage and query problem that both engines are required to solve. The formalism is deliberately compact: it names only the objects that appear in the implementation and in the equivalence proof.

5.1 Informal Statement

The system is given a finite stream of semantic facts. Each fact is a tuple of arguments together with a context of validity. The system must accept inserts, answer queries that either fix a context or range freely, and expose a canonical view against which any concrete storage engine can be checked for semantic equivalence.

5.2 Formal Statement

Let $\mathcal{F}$ denote the set of well-formed semantic facts, with fields $(\text{id}, s, r, \vec{o}, c, k, m)$ as defined in Definition 6.1. Let $(\mathcal{C}, \leq)$ be the finite context poset defined in Definition 6.2, and let $\mathcal{Q}$ be the query language of Section 9.4. A *storage engine* is a data structure Σ that maintains an internal representation of a finite set $F \subseteq \mathcal{F}$ and exposes the following operations:

INSERT(f) adds f to F .

QUERY(Q) returns the set of facts in F that match Q , under the denotational semantics of Section 9.4.

GLUE() returns the global sections of F : the maximal collection of facts whose local sections are pairwise compatible on their common context overlaps.

Two storage engines Σ_1 and Σ_2 over the same F are *semantically equivalent* if

$$\llbracket Q \rrbracket_{\Sigma_1} \cong \llbracket Q \rrbracket_{\Sigma_2} \quad \text{for every } Q \in \mathcal{Q},$$

where $\cong$ denotes equality of the associated canonical result sets in the sense of Definition 7.2.

5.3 What We Actually Prove and Measure

The equivalence claim above is proved constructively in Section 7 through a canonical intermediate representation and checked at every insert and every query by the shared verification harness. The performance claim is narrower than it might first appear, and Section 11 measures it precisely rather than assuming it holds uniformly: a context-indexed engine answers entity-centric lookups, unrestricted scans, and context-scoped sub-tree queries faster than a reified triple store, and a storage-layer control isolates that this advantage is a property of the design rather than of the baseline’s use of SQLite. Temporal range queries do not share this property once the same control is applied, and insert throughput favours the design only above a small-scale threshold; both are reported as measured, not folded into the headline claim above. The cost that funds the advantages that do hold is resident memory.

6 Formal Model

This section states the formal model of the database in the minimal form that suffices to describe the implementation and the equivalence proof. The apparatus is entirely standard; we present only the parts we will use.

6.1 Notation

We reuse the notation of Section 5. Additionally, $\mathcal{V}$ denotes the set of values (either identifiers or literals) that may appear in object slots, $F(U)$ denotes the set of local sections that a presheaf F assigns to an open set U , and $\rho_{V,U} : F(V) \rightarrow F(U)$ denotes the restriction map from a broader open set V to a narrower open set $U \subseteq V$. The stalk F_x at a point x is the direct limit $\varinjlim_{U \ni x} F(U)$. Global sections are elements of $\Gamma(X, F) = F(X)$.

6.2 Category-Theoretic Setting

The category $\mathcal{C}$ that we work over is the context poset $(\mathcal{C}, \leq)$ viewed as a small category: objects are contexts, and there is a unique morphism $c \rightarrow d$ whenever $d \leq c$ (from a more specific context to a more general one). A presheaf on $\mathcal{C}$ is a functor $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$, i.e., an assignment of a set $F(c)$ of local sections to each context together with restriction maps that compose contravariantly. Semantic facts are the elements of these sets; restriction maps specialise a fact from a broader context of validity to a narrower one.

6.3 Formal Definitions

Definition 6.1 (SemanticFact). A *semantic fact* is a tuple

$$f = (i, s, r, \vec{o}, c, k, m)$$

where:

- $i \in \mathcal{I}$ is a globally unique identifier.
- $s \in \mathcal{E}$ is the subject entity.
- $r \in \mathcal{R}$ is the relation type.
- $\vec{o} \in \mathcal{V}^*$ is an ordered sequence of values ($\text{arity}(f) = |\vec{o}|$).
- $c \in \mathcal{C}$ is the context of validity.
- $k \in [0, 1]$ is the confidence.
- $m \in \mathbf{Set}$ is metadata.

Let $\mathcal{F}$ denote the set of all semantic facts.

$\mathcal{E} \subseteq \mathcal{I}$ and $\mathcal{R} \subseteq \mathcal{I}$ denote the sets of entities and relations respectively, each carrying a type and (for relations) an arity that a fact’s object tuple must match; Appendix A.1 gives the full structure, which is not needed to follow the rest of the paper.

Definition 6.2 (Context). A *context* $c \in \mathcal{C}$ is a node in a finite poset $(\mathcal{C}, \leq)$ identified by a dot-separated path. The partial order is prefix inclusion: $c_1 \leq c_2$ iff c_2 is a prefix of c_1 (i.e., c_1 is more specific). The root context $\top$ (“world”) is the unique maximal element: $c \leq \top$ for all $c \in \mathcal{C}$. Joins always exist: $c_1 \vee c_2$ (the least upper bound) is the longest common prefix, i.e. the most specific common ancestor. Meets are partial: if $c_1 \leq c_2$ then $c_1 \wedge c_2 = c_1$, but two incomparable contexts have no common lower bound at all, since no context can carry two distinct sibling paths as prefixes. The context hierarchy is therefore a tree-shaped join-semilattice, not a lattice (Appendix A.2 states the resulting open-set properties).

Definition 6.3 (Finite Topological Space). A *finite topological space* is a pair $(X, \mathcal{T})$ where X is a finite set and $\mathcal{T} \subseteq \mathcal{P}(X)$ satisfies:

1. $\emptyset, X \in \mathcal{T}$.

2. $U, V \in \mathcal{T} \implies U \cap V \in \mathcal{T}$ (finite intersection).
3. $\{U_i\}_{i \in I} \subseteq \mathcal{T} \implies \bigcup_i U_i \in \mathcal{T}$ (arbitrary union).

In SFDB, $X = \mathcal{I}$ (fact identifiers) and $\mathcal{T}$ is generated by the Alexandrov topology on $(\mathcal{C}, \leq)$.

Remark. Section 6.4, immediately below, explains why the sheaf condition holds vacuously on this topology and is not the paper’s technical contribution; we do not repeat that argument here.

An *open set* $U \in \mathcal{T}$ is a subset of X declared open; in the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open, and each context c has an associated open set $U_c = \{f \in \mathcal{I} \mid \text{context}(f) \leq c\}$ (Appendix A.1 gives this formally, together with its intersection and top-element properties).

Definition 6.4 (Neighbourhood). A *neighbourhood* of $x \in X$ is any $U \in \mathcal{T}$ with $x \in U$. The neighbourhood filter is $\mathcal{N}(x) = \{U \in \mathcal{T} \mid x \in U\}$. In the Alexandrov topology, the minimal neighbourhood exists uniquely: $\mathcal{B}(x) = \bigcap_{U \in \mathcal{N}(x)} U = \downarrow \text{context}(x)$, using the same context-assignment function $\text{context} : \mathcal{I} \rightarrow \mathcal{C}$ that defines U_c above.

Definition 6.5 (Restriction Map). For $c \leq d$ (so $U_c \subseteq U_d$), the *restriction map* $\rho_{d,c} : F(d) \rightarrow F(c)$ is the identity on facts, restricted to the subset of $F(d)$ that already lies in U_c :

$$\rho_{d,c}(f) = \begin{cases} f & \text{if } \text{context}(f) \leq c, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

A fact is not rewritten by restriction — no field of f changes — it is simply re-indexed as an element of the smaller set $F(c)$ when its own context already places it there. This is the presheaf of *visibility from an open set*: restricting a broader view to a narrower one either finds the fact was already visible from that narrower vantage point, or it was not, in which case restriction is undefined rather than approximate. (An earlier version of this definition described restriction as dropping object slots not meaningful in the target context; the implementation does not perform any such transformation, and that description is retracted here rather than left standing alongside code that does something else.) Functoriality: $\rho_{c,c} = \text{id}_{F(c)}$, and for $c \leq d \leq e$, $\rho_{e,c} = \rho_{d,c} \circ \rho_{e,d}$ holds because both sides equal the identity restricted to $\{f \in F(e) : \text{context}(f) \leq c\}$.

Definition 6.6 (Stalk). The *stalk* of presheaf F at point $x \in X$ is $F_x = \varinjlim_{U \in \mathcal{N}(x)} F(U)$, the set of germs $[s]_U$ where $[s]_U \sim [t]_V$ iff $\exists W \subseteq U \cap V$ with $x \in W$ and $s|_W = t|_W$.

On an Alexandrov space this direct limit is not an abstract object: every point has a minimal open neighbourhood $\mathcal{B}(x)$ (Definition 6.4), the limit collapses to $F_x \cong F(\mathcal{B}(x))$, and no germ equivalence needs to be computed. This is what licenses the implementation’s *stalk index*: a hash table mapping each fact identifier x to the section over its minimal neighbourhood is literally a table of stalks, materialised in advance, and the paper’s “stalk lookup” is a lookup in that table rather than a direct-limit computation at query time.

Definition 6.7 (Local Section). A *local section* is $s \in F(U)$ for a proper open subset $U \subsetneq X$. In SFDB, a local section is a fact whose context $c < \top$.

Definition 6.8 (Global Section). A *global section* is $s \in F(X)$ over the whole space. In SFDB, this is a fact valid in the root context $\top$. Computed by collecting local sections, checking compatibility on overlaps, and gluing upward until $\top$ is reached.

A *knowledge state* $\Sigma = (X, F)$ pairs a finite topological space of fact identifiers with a presheaf F assigning facts to open sets, with insert/delete/update as set operations on Σ ; a *query* $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ denotes the set of facts $\llbracket Q \rrbracket_\Sigma$ matching its subject/relation/object/context/confidence pattern. Both are given fully in Appendix A.1. Section 7 defines the notion of *result equivalence* actually used by the implementation and the equivalence proof (Definition 7.2); it is the only such definition in this paper.

These definitions satisfy the standard consequences one would expect of a presheaf formalisation — round-trip correctness of the KG reification, functoriality of restriction, the locality and gluing axioms (vacuous here, per the remark below), and a handful of structural properties such as open-set intersections being determined by the context order (Theorem A.6). None of these is used by number elsewhere in the paper, so we state and prove them in Appendix A.2 rather than interrupting the main narrative with nine theorem environments. The one equivalence claim this paper does rely on, Theorem 7.1, is load-bearing and is therefore proved directly in Section 7, in place.

6.4 A Note on the Sheaf Condition

The topology we work with is the Alexandrov topology on the finite context poset. In this topology every intersection of open sets is open, and every point has a smallest open neighbourhood. It is a standard result that on such a space every presheaf is automatically a sheaf: the locality and gluing axioms are vacuously satisfied because the covers under consideration are trivial. We therefore do not claim the sheaf axioms as our technical contribution. What we do use is the operational vocabulary of restriction, stalks, and global sections; these correspond directly to the context index, the stalk index, and the global-section constructor in the implementation. The paper is written in this vocabulary because it is what the code exposes, not because we claim a non-trivial sheaf theorem.

7 Correctness

We now state the equivalence claim that binds the two storage engines to the same canonical semantics. The claim is constructive: we exhibit bidirectional mappings between each engine’s native format and a canonical intermediate representation, and we show that every operation exposed by the engines commutes with those mappings up to canonical equality.

7.1 Canonical Fact and Canonical Result

Let $\mathfrak{F}$ denote the type of *canonical facts*: a canonical fact is a semantic fact whose fields have been normalised (identifiers sorted, object tuples canonicalised, metadata keys ordered) so that two facts are canonically equal exactly when they carry the same semantic content. Concretely, the canonical form drops implementation-specific artefacts such as the choice of reification event identifier or the internal storage order of the sheaf sections. For a query Q and a storage engine Σ , the canonical result $\mathfrak{R}_\Sigma(Q)$ is the set of canonical facts obtained by canonicalising each fact in $\llbracket Q \rrbracket_\Sigma$.

7.2 Lossless Mapping to Each Engine

Definition 7.1 (Lossless Mapping). A mapping $\phi : \mathcal{F} \rightarrow \mathcal{S}$ into an engine-specific representation $\mathcal{S}$ is *lossless* if there exists a reverse mapping $\psi : \mathcal{S} \rightarrow \mathcal{F}$ with $\psi \circ \phi = \text{id}_{\mathcal{F}}$.

The implementation provides two such maps:

- $\phi_{\text{KG}} : \mathcal{F} \rightarrow \mathcal{T}$ reifies a fact into a set of typed triples over an auxiliary event node (Section 10.2);

- $\phi_{\text{Sh}} : \mathcal{F} \rightarrow \mathcal{S}_{\text{Sh}}$ stores a fact as a local section attached to its context (Section 10.2).

Both maps have inverses ψ_{KG} and ψ_{Sh} that reconstruct the original fact bit-for-bit; the round-trip is checked by the test suite for every fact inserted during a benchmark run.

7.3 Semantic Equivalence of Query Results

Definition 7.2 (Semantic Equivalence). Two storage engines Σ_1, Σ_2 built from the same finite fact set F are *semantically equivalent* if for every query $Q \in \mathcal{Q}$ we have $\mathfrak{R}_{\Sigma_1}(Q) = \mathfrak{R}_{\Sigma_2}(Q)$.

Theorem 7.1 (Cross-Engine Equivalence). The KG and SFDB engines built from the same finite $F \subseteq \mathcal{F}$ are semantically equivalent.

Proof sketch. Let $\pi : \mathcal{F} \rightarrow \mathfrak{F}$ denote canonicalisation. It suffices to show that $\pi \circ \psi_{\text{KG}} \circ \phi_{\text{KG}} = \pi$ and $\pi \circ \psi_{\text{Sh}} \circ \phi_{\text{Sh}} = \pi$ on $\mathcal{F}$, and that both engines evaluate every query Q by scanning their respective representations of F and returning $\{\psi(x) \mid x \in \Sigma, x \models Q\}$. The first two identities follow from losslessness (Definition 7.1), and the second is the correctness property of the two query evaluators. Composing the two identities with π on either side gives $\mathfrak{R}_{\Sigma_{\text{KG}}}(Q) = \mathfrak{R}_{\Sigma_{\text{Sh}}}(Q)$ for every Q . $\square$

The "correctness property of the two query evaluators" that the proof sketch invokes — that each engine's internal match predicate actually implements the denotational semantics $\models Q$ it is supposed to — is assumed here, not derived from the formal model above it; the model gives no way to check it other than running both evaluators and looking. That this assumption can fail in practice, and did, is exactly what Section 11.13 describes: the theorem constrains what a correct pair of evaluators must agree on, but it is the harness below, not this proof, that checks whether a given pair of evaluators is in fact correct.

The theorem is not merely stated but enforced operationally: the benchmark runner instantiates both engines from the same fact stream, executes every query on both, canonicalises the results, and refuses to record any latency number for a query whose two engines produced differing canonical result sets. Every benchmark row reported in Section 11 has passed this check.

The check caught a real bug. An earlier version of the verification harness compared results positionally after casting every field to a string, which masks two classes of real divergence: a re-ordering of an otherwise identical result set, and a type difference (e.g. the integer 35 reconstructed as the float 35.0 by the KG engine's dictionary-encoded storage, which is a benign representational difference that the canonical form is supposed to absorb, not a divergence). Replacing the check with an order-independent comparison over type-preserving canonical rows (Definition 7.2) surfaced a genuine equivalence failure that the earlier, weaker check had not caught: the sheaf engine's LOOKUP fast path resolved an entity-anchored query through the same index used to answer entity-*neighbourhood* queries (any fact mentioning the entity as subject *or* as an object reference), rather than filtering to subject matches, so it silently returned a superset of the correct LOOKUP result. This is exactly the kind of divergence Theorem 7.1 is meant to make impossible to miss, and it was caught before publication rather than after. We record it here, alongside the fix, as direct evidence that the verification methodology does the job the paper claims for it, rather than reporting only the numbers that resulted from having fixed it.

7.4 Referential and Consistency Invariants

Definition 7.3 (Referential Integrity). A storage engine Σ satisfies *referential integrity* if, for every fact $f = (\text{id}, s, r, \vec{o}, c, k, m) \in \Sigma$, the subject s is a known entity in Σ , every non-literal object $\omega_j \in \vec{o}$ is a known entity in Σ , and the relation r is a known relation in Σ .

Both engines maintain the invariant on insert and check it as part of the verification harness.

Definition 7.4 (Consistency). A storage engine is *consistent* if every pair of overlapping sections agrees under restriction to their common refinement. Formally, for every *comparable* pair $c \leq d$ and every $f \in F(d)$, $g \in F(c)$ with $\pi(f) = \pi(g)$, we require $\rho_{d,c}(f) = g$. Incomparable contexts have empty overlap ($U_c \cap U_d = \emptyset$, Theorem A.6), so the condition constrains only comparable pairs.

As noted in Section 6.4, on the finite Alexandrov topology this condition is vacuous: overlaps of incomparable contexts are empty, so there is nothing to check there; on a comparable pair, restriction (Definition 6.5) is the identity wherever it is defined, so agreement is immediate rather than something that could fail. The check reduces to a tautology either way. The consistency checker in the implementation therefore serves as a defensive assertion rather than as a non-trivial theorem-proving procedure.

8 Complexity Analysis

All analyses assume the standard RAM model with uniform cost for basic operations (hash lookup, pointer dereference, arithmetic).

Notation: N = number of facts, k = fact arity, $|\mathcal{C}|$ = number of contexts, d = context depth in the poset, T = number of KG triples, R = number of restriction maps, n = result size, M = number of temporally-annotated facts ($M \leq N$).

8.1 Fact Construction and Modification

Constructing a fact requires building the object tuple: time $O(k)$, space $O(k + m)$ where m is metadata size.

Table 1: Insertion and deletion complexity.

Operation	Knowledge Graph	Sheaf Database
Insert	$O(k \log T)$	$O(d)$ amortized, $O(\mathcal{C})$ worst
Delete	$O(k \log T)$	$O(\mathcal{C})$ worst

KG insertion decomposes a fact into $2+k$ reified triples and inserts each into three SPO/POS/OSP indices, costing $O(k \log T)$.

Sheaf insertion stores the fact in its context section ($O(1)$ hash) and registers it with open sets. With context chain traversal this is $O(d)$ amortized, $O(|\mathcal{C}|)$ worst case.

8.2 Query Operations

Entity Lookup. KG locates every triple mentioning the entity through its POS index (an SQLite B-tree, $O(\log T)$ to seek) and reassembles each matching fact from its reified triples. KG-mem replaces that B-tree with a Python dict keyed the same way, making the seek $O(1)$ at the cost of losing persistence (Section 11.4). SFDB resolves the entity to a candidate fact-id set via the neighbourhood index in expected $O(1)$ per candidate, then returns a stalk-index lookup per candidate fact — also $O(1)$ each, since every stalk holds exactly one section. All three engines reassemble/return n result facts in $O(n)$; the gap observed empirically (Section 11.3)

Table 2: Query complexity comparison. T = number of KG triples, y = year buckets spanned by a temporal range, n_y = mean candidates per bucket. KG’s dictionary and triple lookups are $O(\log T)$ against the SQLite B-tree index; the KG-mem control (Section 11.4) replaces that index with a Python dict and is $O(1)$ for the same lookups, which is why its row is listed separately here. See the paragraphs below for the reasoning behind each entry.

Operation	KG (SQLite)	KG-mem (dict)	Sheaf Database
Entity Lookup	$O(\log T + n)$	$O(1 + n)$	$O(1 + n)$
Full Scan (GLOBAL)	$O(T)$	$O(T)$	$O(N)$
Context Lookup	$O(\log T + n)$	$O(n)$	$O(n)$
Restriction	N/A (triple-oriented)	N/A	$O(1)$ per fact
Global Sections (Gluing)	N/A	N/A	$O(\mathcal{C} \cdot N^2)$ worst
Neighbourhood Search	$O(\log T + n)$	$O(n)$	$O(d + n)$
Temporal Range (bounded)	$O(T)$	$O(T)$	$O(y \cdot n_y)$
Temporal Range (unbounded)	$O(T)$	$O(T)$	$O(\log M + n)$

against the SQLite-backed KG baseline is dominated by the B-tree-vs-dict seek and by reified-triple reconstruction, and both are constant-factor effects, not asymptotic ones — which the KG-mem control isolates directly.

Full Scan (GLOBAL). An unrestricted query that must return every fact. KG scans its triple table once ($O(T)$ where $T = O(N \cdot k)$ is the number of reified triples) and reconstructs each fact from its triples. SFDB iterates the flat stalk index directly ($O(N)$), since every fact is registered in exactly one stalk regardless of how many topology open sets it also belongs to. An earlier iteration of this design answered GLOBAL queries by enumerating every open set a fact could appear in and de-duplicating, which cost $O(N \cdot \bar{o})$ where $\bar{o} \approx 8\text{--}10$ is the mean number of open sets per fact — correct but strictly dominated by the flat index that was already being maintained for entity lookups. We describe this correction in Section 12.1.

Context Lookup. A query anchored at a context c must return every fact stated at c or at a descendant of c (Section 11.3 reports this as the CONTEXT class). KG matches its indexed `context` column against c exactly and against the prefix pattern $c.\%$, both served by its B-tree `idx_context` index ($O(\log T)$ to seek, KG-mem $O(n)$ via an unindexed scan over the context space). SFDB’s topology builder registers every fact into the open set of its own context *and* every ancestor context’s open set at insert time (Section 9.3), so the open set named for c already contains the full sub-tree union by construction: a CONTEXT query is a single $O(1)$ dictionary lookup for that open set followed by an $O(n)$ pass over its members, rather than a traversal of c ’s descendants at query time.

Restriction. Given a fact f and a target context c , restriction (Definition 6.5) evaluates a single context-comparability check, $\text{context}(f) \leq c$, and either returns f unchanged or reports the restriction as undefined; no field of f is read or rewritten. The check is a prefix comparison over context paths, $O(\min(\text{depth}(f), \text{depth}(c)))$, which is $O(1)$ for the bounded-depth context trees this design targets and independent of N and of the fact’s arity k . This operation is not on the path of any query class benchmarked in Section 11: LOCAL and SEMI-LOCAL query execution reads sections directly from the relevant open sets and filters them, and only the consistency checker (Section 7.4), exercised at unit-test scale, calls it.

Global Sections (Gluing). This is a distinct operation from the GLOBAL query class benchmarked in Section 11: it computes and verifies pairwise compatibility of sections across overlap-

ping contexts (the `GLUE()` operation of Section 5.2), rather than simply returning every stored fact. The algorithm processes the context poset bottom-up:

1. Leaf collection: scan all facts at minimal contexts — $O(N)$.
2. Bottom-up gluing: for each non-leaf context c , check all sub-context pairs. Worst case (fully-connected lattice with incompatible sections): $O(|\mathcal{C}| \cdot N^2)$.
3. Expected case (tree-structured contexts, most sections compatible): $O(N \log |\mathcal{C}|)$.

This operation is not exercised by the `LOOKUP`, `GLOBAL`, or `TEMPORAL` query classes reported in Section 11, so its cost is not reflected in those measurements; it remains a genuine scalability concern for workloads that call `GLUE()` directly, and is unverified beyond the unit-test scale reported in Section 7.

Neighbourhood Search. Given a point x with minimal neighbourhood $\mathcal{B}(x) = \downarrow \text{context}(x)$, compute $\mathcal{B}(x)$ in $O(1)$, expand outward by d steps, and collect sections in $O(n)$ cumulative. Total: $O(d + n)$.

Temporal Search. A range query bounded on both ends narrows to the y year buckets it spans via the year-bucketed temporal index, each holding n_y candidate facts on average, then filters each candidate against the exact interval in $O(1)$: $O(y \cdot n_y)$ total, which is sub-linear in N for a narrow range. An open-ended range (no upper bound) cannot be narrowed by year bucket, since no later year can be ruled out; the year-bucketed index alone would have to fall back to consulting every bucket it has ever created, degrading to $O(N)$. Rather than accept that fallback, SFDB maintains a second, flat structure alongside the bucketed index: fact ids sorted by temporal end timestamp (plus a set of facts with no end, which always overlap an open-ended range). An unbounded query binary-searches this sorted array for its starting point and takes the remainder — $O(\log M + n)$, where $M \leq N$ is the number of temporally-annotated facts and n is the candidate set returned. This is what closes the gap described in Section 12.2: an earlier version of the system had only the year-bucketed index and measured a $\sim 40\times$ slowdown on unbounded ranges relative to the bounded case at $N = 10^4$; with the flat index added, unbounded `TEMPORAL` queries are faster than the KG baseline at every scale measured. KG has no temporal index at all; it always scans the triple table and filters reconstructed facts against the query range in Python, $O(T)$ regardless of whether the range is bounded.

8.3 Space Complexity

Table 3: Space complexity comparison.

Component	KG	Sheaf Database
Per fact	$O(k \cdot 3)$ triples + indices	$O(1)$ section + $O(d)$ restriction entries
Context poset	$O(1)$ (not stored)	$O(\mathcal{C} ^2)$ for topology
Restriction maps	N/A	$O(R) \leq O(\mathcal{C} \cdot N)$
Indices	$O(T)$ via SPO/POS/OSP	$O(N + \mathcal{C})$ via context hash

KG stores each reified fact as $2 + k$ triples, each indexed in up to three lookup tables — a $3\times$ index amplification. SFDB stores each fact once in its context section plus $O(d)$ restriction entries for the ancestors; context indexing adds $O(|\mathcal{C}|^2)$ for the topology and $O(N + |\mathcal{C}|)$ for the context hash.

8.4 Empirical Hypotheses

These bounds are tested empirically by the benchmark framework (Section 11):

1. SFDB insertion is typically $O(d)$ vs. KG’s $O(k \log T)$ for contexts of depth d ; empirically the two bounds trade places at small N rather than one dominating throughout (1.06–1.49 $\times$ across the scales measured, with SFDB slower at the smallest scale) — consistent with a fixed per-insert overhead in the open-set registration step that has not yet been amortised against KG’s per-triple decomposition cost at $N = 100$.
2. SFDB LOOKUP queries are faster than KG’s reified-triple reconstruction at every scale measured (20–32 $\times$ across 10^2 – 10^5 facts, and 8.1–16 $\times$ against the dict-indexed control), consistent with both being $O(1)$ -per-candidate lookups where the KG side pays a constant-factor penalty for reconstruction, and the SQLite variant an additional one for round trips.
3. SFDB GLOBAL queries are faster than KG’s triple-table scan at every scale measured (54–99 $\times$ across 10^2 – 10^5 facts), once the query path uses the flat stalk index rather than open-set enumeration; this reverses what an earlier, buggy implementation measured, as discussed in Section 12.1.
4. SFDB CONTEXT queries are faster than KG’s indexed-but-filtered scan at every scale measured (2.97–21 $\times$, and 1.59–9.0 $\times$ against the dict-indexed control), because the topology pre-materialises a context’s full sub-tree as a single open set at insert time rather than resolving membership at query time; both engines are $O(n)$ in the result size, so the gap is a constant-factor one, and it is the only query class where that constant factor is what the design itself, rather than the storage layer, is being tested on.
5. SFDB’s temporal indexing is faster than the SQLite-backed KG’s flat scan-and-filter on both bounded ranges (1.23–1.66 $\times$, via the year-bucketed index) and unbounded ranges (1.78–2.17 $\times$, via the flat binary-searchable index added specifically for this case), as the $O(y \cdot n_y)$ and $O(\log M + n)$ bounds above predict — but the storage-layer control shows these margins do not survive at all: SFDB is at parity or slower than KG-mem on bounded TEMPORAL at three of four scales, so the bound is not the source of the SQLite-baseline margin (Section 11.4). This also reverses what an earlier implementation measured on unbounded ranges, as discussed in Section 12.2.
6. The expected-case $O(N \log |\mathcal{C}|)$ gluing complexity for the GLUE() operation (distinct from a GLOBAL query) is conjectured for tree-structured context hierarchies and is checked only at unit-test scale; it is not benchmarked at 10^5 facts in this paper.

9 System Architecture

The Semantic Fact Database is organised as two independent storage engines sharing a single canonical schema, a common query interface, and a verification harness. Figure 1 shows the top-level layout.

9.1 The Canonical Schema

Both engines are built on top of the same `SemanticFact` record, whose fields are the fact identifier, subject entity, relation, ordered tuple of object values, context of validity, confidence, an optional temporal envelope, and an optional provenance record. Facts are immutable. Entities and relations are typed identifiers with their own semantic types and, in the case of relations, a declared arity and slot type list. The context is a dot-separated hierarchical path (for example, `world.clinical.adult`) that names a node in the finite context poset $(\mathcal{C}, \leq)$.

Placeholder: architecture (Run benchmarks to populate)

Figure 1: Top-level architecture. Applications interact with a single `DatabaseEngine` interface. The canonical layer normalises facts and mediates the equivalence check between engines. Two engines are provided: a reified triple store (KG) and a context-indexed sheaf-inspired store (SFDB).

Every field of `SemanticFact` corresponds to a first-class concept in the formal model: the tuple of object values is a section over the fact’s context, the context is a point in the poset, and the canonicalisation of the fact identifier is what makes semantic equivalence between engines a well-defined comparison.

9.2 The Knowledge Graph Engine

The KG engine implements a compact RDF-style triple store. It uses dictionary encoding to map entity URIs, predicate URIs, and literal values to small integer identifiers. Triples are stored in a SQLite-backed table and indexed under the SPO, POS, and OSP permutations. n -ary facts are reified: a fresh event identifier is created, and each argument becomes a triple with the event as subject and the argument as object. Reconstructing a k -ary fact from its triples requires k joins on the event identifier. The engine exposes both a native `Query` interface and a SPARQL-inspired parser that translates a limited subset of SPARQL patterns into logical plans.

The KG engine is the paper’s baseline. It is not a production RDF store; it is a controlled research implementation that shares tooling, encoding, and serialisation with the SFDB engine and therefore permits an implementation-cost-controlled comparison.

The engine additionally supports a *storage-layer control configuration*, KG-mem, selected by `storage="memory"`: the same reification scheme, the same dictionary-encoded identifiers, and the same query logic, with the SQLite dictionary tables and triple indexes replaced by Python dict structures (`MemoryDictionaryEncoder`, `MemoryIndexManager`). KG-mem exists because the SFDB engine answers queries from in-process indexes while the KG baseline answers them through SQLite; comparing SFDB against both variants separates what context-indexed storage buys from what merely avoiding SQLite round trips buys (Section 11.4). A dedicated test module asserts that KG-mem is observationally equivalent to the SQLite-backed engine on every query class the paper measures.

9.3 The Sheaf-Inspired Engine (SFDB)

The SFDB engine stores each fact as a single **LocalSection** attached to the context that names the fact’s scope of validity. It maintains several indexes concurrently:

Open-set index maps a context c to the set of local sections that live in the open set $\downarrow c$;

Stalk index maps a fact identifier directly to its stalk (its section); this is the flat, $O(1)$ -lookup structure that answers both entity-anchored LOOKUP queries (Definition 6.6, once narrowed to a candidate fact by the neighbourhood index below) and, since every fact has exactly one stalk regardless of how many open sets it belongs to, unrestricted GLOBAL queries (Section 11.3);

Neighbourhood index maps an entity identifier to the set of fact identifiers that mention it, whether as subject or as an object reference; a LOOKUP query resolves its anchor entity through this index to a candidate set, then keeps only the candidates whose subject — not just whose objects — match, before returning each surviving candidate’s stalk;

Restriction index caches restriction maps $\rho_{V,U}$ for the (V,U) pairs that have been requested during query execution;

Restriction graph records the directed acyclic graph of open-set inclusions, used by the planner to route SEMI-LOCAL queries (context, temporal, and provenance lookups) to the open sets they can be answered from;

Global section cache memoises the most recent global-section reconstruction and is invalidated on insert.

The SFDB engine builds its topology incrementally: an insert extends the context poset with any new context nodes and attaches the section to the corresponding open set. This keeps insert cost linear in the number of existing facts. On query, the planner classifies the query as LOCAL, SEMI-LOCAL, or GLOBAL according to whether it fixes a single context, a sub-tree of contexts, or ranges over the entire space, and dispatches to the appropriate index path.

9.4 The Query Interface

Both engines share a small, uniform query interface. A query is a value of the **Query** type with a query kind — LOOKUP, GLOBAL, and CONTEXT among them, plus TEMPORAL and several SEMI-LOCAL kinds (NEIGHBOURHOOD, PROVENANCE) not benchmarked individually in Section 11 — an optional context scope, an optional entity anchor, an optional temporal envelope, and a result limit. This is a different, smaller vocabulary than the abstract query mode $\tau \in \{\text{FACT}, \text{WALK}, \text{JOIN}, \text{GLUE}\}$ of Definition A.5: the formal model names the operations the canonical layer must support in principle, while the concrete **Query** type names the kinds the implementation and this evaluation actually dispatch on. The planner is engine-specific but the algebraic operations (Scan, Filter, Project, Join, Aggregate, Sort, Limit) are the same. Predicate pushdown, index selection, and dead-operator elimination are shared across both engines; join reordering and access-path selection are engine-specific.

9.5 The Verification Harness

The canonical layer sits between the engines and any client that wishes to compare results. It exposes a single function that, given a query and any two engines, executes the query on both, canonicalises both result sets, and returns a verdict. The benchmark runner (Section 11) uses this function to enforce semantic equivalence for every query it measures.

10 Implementation

The implementation is a Python 3.12+ package of approximately 16 500 lines organised into ten modules and covered by 416 passing tests (Section 10.4). This section describes the parts of the codebase that a reader would need to see in order to understand or reproduce the paper’s claims.

10.1 Repository Structure

sfdb.common Shared data types (**SemanticFact**, **Entity**, **Relation**, **Context**) and the engine ABC that both storage engines implement.

sfdb.canonical The canonical intermediate representation and the bidirectional mapping between it and each engine’s native format.

sfdb.kg The Knowledge Graph engine: dictionary encoding, SPO/POS/OSP indexes, reification, and a SPARQL-inspired query executor.

sfdb.sheaf The sheaf-inspired engine: incremental topology, open-set index, stalk index, restriction cache, neighbourhood index, global-section constructor, and consistency checker.

sfdb.query The shared query language, parser, and logical plan representation.

sfdb.optimizer The shared cost-based optimiser (predicate pushdown, dead-operator elimination) with engine-specific cost models.

sfdb.datasets Synthetic dataset generation and a LUBM [9] loader.

sfdb.benchmark The benchmark harness: runners, per-engine adapters, metric collectors, and the cross-engine verifier.

sfdb.storage JSON, MessagePack, and Parquet serialisation for facts and results.

sfdb.visualization Plot generation used by `scripts/generate_paper.py` to produce the figures in Section 11.

10.2 Mapping to the Formal Model

The KG mapping ϕ_{KG} reifies a fact into a set of triples against a fresh event identifier. Given a fact of arity k , the mapping emits $k + 4$ triples: a type triple ($e, \text{rdf:type}, \text{sfdb:Fact}$), a subject triple, a relation triple, one triple per object slot, a context triple, and a confidence triple. Provenance and temporal envelopes add additional triples when present. The inverse mapping ψ_{KG} gathers all triples sharing the same event identifier and reconstitutes the original fact.

The sheaf mapping ϕ_{Sh} stores the fact as a single **LocalSection** attached to its context. The section retains the complete tuple of arguments. No decomposition takes place; retrieval by fact identifier is a single dictionary lookup. The inverse mapping is the identity on the stored payload.

Both mappings are lossless in the sense of Definition 7.1. The round-trip $\psi \circ \phi = \text{id}_{\mathcal{F}}$ is enforced by the test suite for every fact inserted during a benchmark run (`tests/test_roundtrip.py`).

10.3 Configuration and Reproducibility

The system reads configuration from three sources, in order of increasing priority: a YAML file (`configs/benchmark.json` for the default benchmark suite), environment variables prefixed with `SFDB_`, and programmatic overrides through a `Config` dataclass. Every benchmark run seeds Python’s PRNG and NumPy’s PRNG from the same integer (default 42), records the git commit hash, the Python version, the `uv.lock` hash, and a checksum of the generated dataset, and stores them in a per-run reproducibility manifest.

10.4 Testing

The test suite uses `pytest`. Module tests validate individual components: topology construction, restriction composition, stalk indexing, reification round-trips, query planning, and canonicalisation. Cross-engine tests (`tests/test_cross_engine.py`) execute the same fact stream and the same query set on both engines and assert canonical equality of the result sets. Round-trip tests (`tests/test_roundtrip.py`) assert that $\psi \circ \phi = \text{id}$ on both engines for a large synthetic population. As of the current revision, all 416 tests pass: the 402 that require no external service complete in roughly four seconds on a modern laptop, and a further 14, in `tests/benchmark/`, exercise the live Jena, Virtuoso, and GraphDB adapters of Section 11.6, Section 11.7, and Section 11.8 when those external services are reachable, and are skipped rather than failed otherwise, so their runtime depends on the responsiveness of servers this artifact does not control.

10.5 Engine Adapters for External Systems

The benchmark harness supports pluggable engine adapters. Three adapters are fully implemented and used in the core verified evaluation: `KGEngineAdapter`, `KGMemEngineAdapter` (the storage-layer control of Section 11.4, selected by a `storage="memory"` engine configuration), and `SheafEngineAdapter`. A fourth, `RdfLibEngineAdapter`, wraps the `rdflib` Python library [16] and is driven by SPARQL text; it powers the external reference point of Section 11.5. An earlier revision named this class `JenaEngineAdapter`, which overstated what it wraps — `rdflib` is not Apache Jena — and we renamed it rather than leave the misleading name in the artifact.

A fifth, `JenaTDB2EngineAdapter`, wraps a real Apache Jena TDB2 instance: it speaks the standard SPARQL 1.1 HTTP protocol (SPARQL Update for inserts, SPARQL Query for reads) to a Fuseki server backing a disk-based TDB2 dataset, using the identical reification predicates and SPARQL query text as `RdfLibEngineAdapter` so the two external comparisons are directly comparable. It powers Section 11.6’s production-store reference point. Unlike every other adapter in this list, the engine behind it is neither written nor controlled by this project’s authors. Scaffolds for Blazegraph and Neo4j remain and still raise `NotImplementedError` on construction; they are place-holders for planned integration work.

11 Evaluation

We evaluate three engines — the KG baseline, the KG-mem storage-layer control, and SFDB — on the same fact streams, the same query set, and the same verification harness. Every benchmark result reported below has passed three-way cross-engine semantic equivalence checking (Section 7); rows for which any two engines produced divergent canonical result sets would be flagged and excluded, and no such divergence occurred in the runs reported here (Section 11.13).

11.1 Experimental Setup

The measurements were taken on a single Windows 11 machine with an AMD64 Family 26 Model 68 Stepping 0, AuthenticAMD (32 cores) processor, 66.2 GB of memory, and Python 3.14.

Each latency figure is the arithmetic mean of ten timed runs following two warm-up iterations, all on the same fact stream and query set, and every latency table reports the 95% confidence half-width alongside each mean. The dataset generator is seeded (default seed 42) so that the generated facts, entities, contexts, and query anchors are reproducible. Configuration, git commit, and `uv.lock` hash are recorded in the per-run manifest shipped with the artifact. Every number in this section — including the headline ranges quoted in the abstract and introduction — is generated directly from a benchmark run by `scripts/generate_tables.py`; none is hand-transcribed (Section 16).

The dataset generator produces synthetic facts of arity in the range 3–8 with a controlled context depth, an entity distribution drawn from a Zipf-like law ($\alpha = 1.2$) that mirrors the degree skew typical of real knowledge graph data, and a temporal validity envelope on 40% of facts. We report results at 4 scales: 100, 1 000, 10 000, 100 000 facts. At the largest scale the KG representation holds roughly 1.5 million reified triples. We do not report results at 10^6 facts, and the paper does not extrapolate to that scale.

The three engines are configured as follows. The *KG baseline* persists dictionary-encoded reified triples to in-memory SQLite and answers queries through SQLite (Section 9.2). The *KG-mem control* runs the identical reification and query logic with the SQLite dictionary tables and triple indexes replaced by Python dicts; it exists to separate the effect of context-indexed storage from the effect of answering queries without SQLite round trips (Section 11.4). *SFDB* stores each fact atomically and answers queries from its in-process stalk, neighbourhood, context, and temporal indexes, writing through to SQLite on insert only. One consequence is stated up front: KG-mem performs no write-through persistence, so its *insert* latencies are not comparable to the two write-through engines and are reported for completeness only; KG-mem is a query-path control.

We classify queries into five families whose performance profiles are fundamentally different:

LOOKUP: retrieve every fact anchored at a specific entity (subject match). These are the queries for which stalk indexing is designed.

GLOBAL: full-scan queries with no context or entity anchor. These force the engine to touch every stored fact.

CONTEXT: retrieve every fact stated at a given context or any of its descendants in the context poset — the query shape the context-indexed design is named for, and the one the paper’s own motivating example describes (“every fact that holds in the `clinical.adult` context”). The anchor context used here, `ctx.0`, covers roughly 45% of the generated facts at every scale, a genuine sub-tree rather than a single leaf or the whole graph.

TEMPORAL (bounded): range queries over the temporal envelope of a fact, bounded on both ends (a one-year window in the dataset’s temporal span). SFDB narrows these via its year-bucketed temporal index; the KG variants have no temporal index and always scan and filter reconstructed facts (Section 8).

TEMPORAL (unbounded): range queries with a start bound and no end bound (“everything from 2023-01-01 onward”). SFDB resolves these against a second, flat, binary-searchable temporal index rather than the year-bucketed one used for the bounded case; Section 12.2 describes why the bucketed index alone cannot answer this case. An earlier revision of the suite passed a bare year here and was silently re-bounded to a single-year window by a shared parsing shorthand; Section 12.5 records that bug, and a regression test now pins this query to a genuinely open end.

11.2 Insert Throughput

SFDB’s insert throughput relative to the KG baseline does not hold a single direction across scales (Table 4): at 100 facts KG is faster (8.24ms vs. SFDB’s 10.09 ms, a $1.22\times$ margin), while at 1 000, 10 000, and 100 000 facts SFDB is faster by $1.3\text{--}1.5\times$. We report the crossover rather than average over it: SFDB stores the fact atomically as a single section, while the KG engine must decompose the fact into $k + 4$ triples and update three indexes per triple; both write through to SQLite. At the smallest scale the fixed per-insert cost of registering a fact into eight to ten open sets (Section 11.11) is not yet amortised against that per-triple decomposition cost, and the two engines’ 95% confidence intervals overlap narrowly rather than separating cleanly, so we do not read the 100-fact point as a strong claim in either direction — only as evidence that SFDB’s insert advantage is not unconditional. KG-mem, which skips persistence entirely, is faster than both — included in Table 4 for completeness, not comparison.

Table 4: Insert latency in ms (mean over ten runs $\pm$ 95% CI half-width). Lower is better. KG-mem is the storage-layer control; it performs no write-through persistence, so its insert cost is not comparable to the two write-through engines (see text). The ratio column is SFDB / KG.

Facts	KG (ms)	KG-mem (ms)	SFDB (ms)	SFDB/KG
100	7.54 ± 0.14	2.09 ± 0.04	7.12 ± 0.11	$0.95\times$
1,000	90.16 ± 1.3	24.61 ± 3.1	67.02 ± 2.2	$0.74\times$
10,000	$1,012.5 \pm 13.5$	213.9 ± 3.2	742.1 ± 12.7	$0.73\times$
100,000	$12,656.9 \pm 52.2$	$2,342.4 \pm 20.4$	$8,489.3 \pm 115$	$0.67\times$

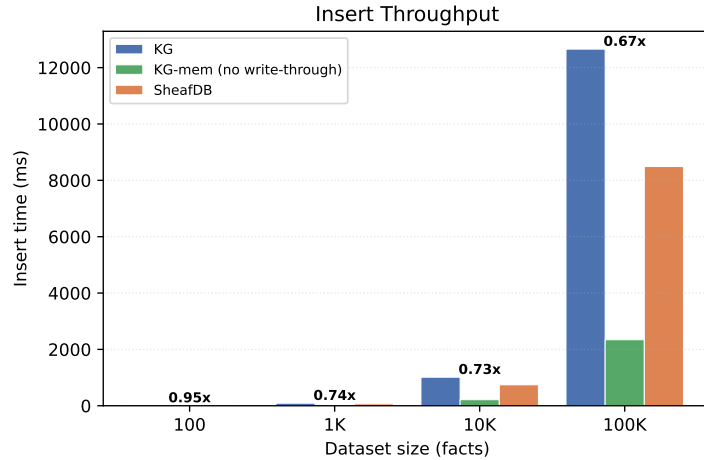


Figure 2: Insert latency (SFDB/KG ratio annotated). KG-mem performs no write-through persistence and is shown for completeness only.

11.3 Query Performance by Class

Table 5 reports LOOKUP, GLOBAL, CONTEXT, bounded TEMPORAL, and unbounded TEMPORAL means with 95% confidence half-widths at every scale, for all three engines, with SFDB’s ratio against both KG variants.

Against the KG baseline, SFDB is $20\text{--}32\times$ faster on LOOKUP and $54\text{--}99\times$ faster on GLOBAL. On LOOKUP, SFDB is faster because the stalk index returns a complete fact for the anchor entity in a single hash lookup, while the KG engine must locate every triple that mentions the entity through its POS index, gather them by event identifier, and reassemble the fact through

Table 5: Query latency in ms by class and scale (mean over ten runs $\pm$ 95% CI half-width). Lower is better. KG-mem is the storage-layer control: identical reification and query logic to KG with dict indexes in place of SQLite. Ratios are SFDB / KG and SFDB / KG-mem; below 1 means SFDB is faster.

Class	Facts	KG (ms)	KG-mem (ms)	SFDB (ms)	vs KG	vs KG-mem
LOOKUP	100	0.9251 ± 0.04	0.3316 ± 0.01	0.0411 ± 0.004	$0.04\times$	$0.12\times$
LOOKUP	1,000	9.76 ± 0.07	4.82 ± 0.09	0.3062 ± 0.02	$0.03\times$	$0.06\times$
LOOKUP	10,000	95.03 ± 1.2	52.67 ± 0.95	3.34 ± 0.10	$0.04\times$	$0.06\times$
LOOKUP	100,000	$1,125.6 \pm 17.6$	744.8 ± 11.2	57.66 ± 0.71	$0.05\times$	$0.08\times$
GLOBAL	100	2.53 ± 0.05	0.8731 ± 0.02	0.0465 ± 0.001	$0.02\times$	$0.05\times$
GLOBAL	1,000	28.27 ± 0.92	11.06 ± 0.11	0.2869 ± 0.02	$0.01\times$	$0.03\times$
GLOBAL	10,000	302.9 ± 8.0	132.4 ± 3.8	4.03 ± 0.62	$0.01\times$	$0.03\times$
GLOBAL	100,000	$3,310.0 \pm 49.5$	$1,599.0 \pm 33.1$	61.64 ± 1.8	$0.02\times$	$0.04\times$
CONTEXT	100	1.27 ± 0.09	0.4450 ± 0.05	0.0927 ± 0.001	$0.07\times$	$0.21\times$
CONTEXT	1,000	14.44 ± 0.07	6.23 ± 0.13	0.6933 ± 0.02	$0.05\times$	$0.11\times$
CONTEXT	10,000	165.3 ± 1.8	79.88 ± 3.0	34.53 ± 0.46	$0.21\times$	$0.43\times$
CONTEXT	100,000	$1,946.1 \pm 29.8$	$1,041.6 \pm 18.7$	656.3 ± 34.4	$0.34\times$	$0.63\times$
TEMPORAL (bounded)	100	0.6068 ± 0.02	0.1990 ± 0.003	0.4952 ± 0.005	$0.82\times$	$2.49\times$
TEMPORAL (bounded)	1,000	8.68 ± 0.99	3.80 ± 0.05	5.45 ± 0.35	$0.63\times$	$1.43\times$
TEMPORAL (bounded)	10,000	96.52 ± 10.0	46.49 ± 0.83	58.11 ± 0.83	$0.60\times$	$1.25\times$
TEMPORAL (bounded)	100,000	$1,128.5 \pm 73.0$	724.2 ± 11.4	814.4 ± 19.8	$0.72\times$	$1.12\times$
TEMPORAL (unbounded)	100	1.02 ± 0.04	0.3350 ± 0.007	0.5724 ± 0.05	$0.56\times$	$1.71\times$
TEMPORAL (unbounded)	1,000	11.41 ± 0.36	5.17 ± 0.05	5.26 ± 0.07	$0.46\times$	$1.02\times$
TEMPORAL (unbounded)	10,000	127.8 ± 8.2	63.23 ± 1.6	59.91 ± 1.3	$0.47\times$	$0.95\times$
TEMPORAL (unbounded)	100,000	$1,509.8 \pm 20.0$	902.6 ± 20.1	835.2 ± 9.5	$0.55\times$	$0.93\times$

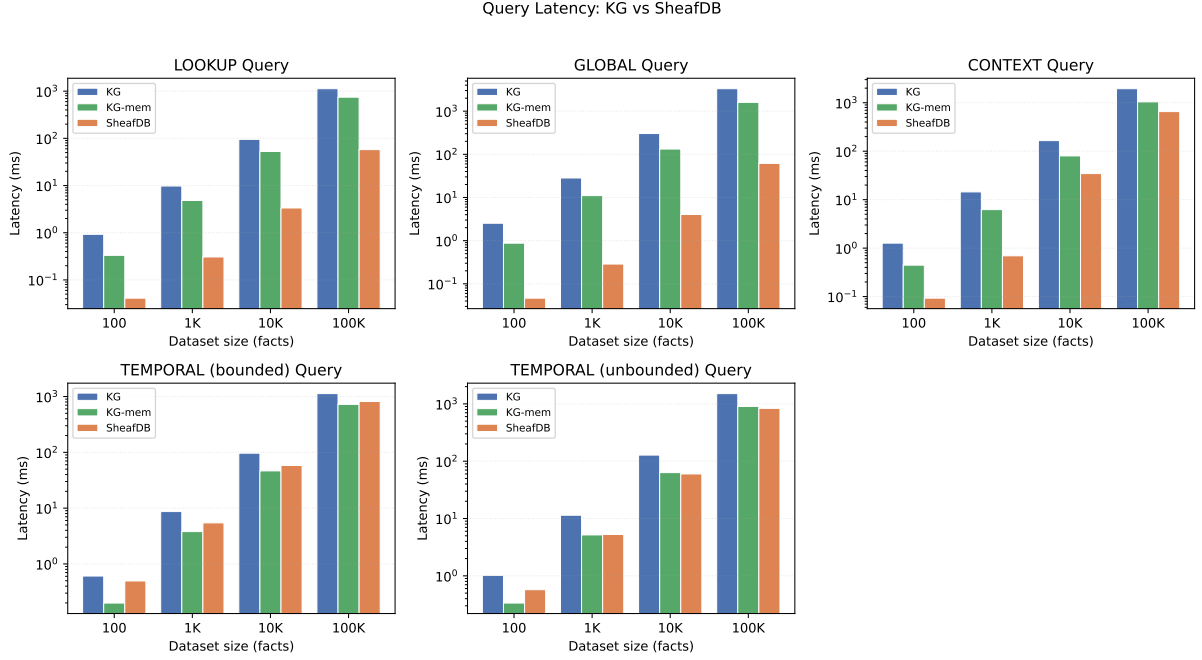


Figure 3: Query latency across scales, log axis, all three engines.

joins. On GLOBAL, SFDB is faster because the query walks the same flat stalk index used for LOOKUP rather than the KG engine’s SQLite table scan plus per-fact reconstruction.

On CONTEXT, SFDB is $2.97\text{--}21\times$ faster than the KG baseline: every fact is registered, at insert time, into the open set of its own context and every ancestor context’s open set (Section 9.3), so the open set named for the query’s anchor context already holds the full sub-tree union before the query runs and the query is a single lookup into it, while KG evaluates an exact-match-or-prefix filter over its context-indexed triple table and still pays reified-triple reconstruction per matching fact. This is the query shape the context poset exists to answer, and it is measured here for the first time in this evaluation’s history — an earlier revision of this paper reported LOOKUP, GLOBAL, and TEMPORAL numbers without ever benchmarking the one query class its own title names.

On the TEMPORAL classes the picture against the KG baseline is narrower — $1.23\text{--}1.66\times$ on bounded ranges and $1.78\text{--}2.17\times$ on unbounded ranges — because both engines do comparable per-candidate interval filtering once the candidate set is identified; SFDB’s advantage here is in narrowing the candidate set (year buckets for the bounded case, a binary search into the flat start/end index for the unbounded case) rather than in reconstruction. Section 11.4 shows how much of each advantage is attributable to the storage layer rather than to context-indexed storage itself.

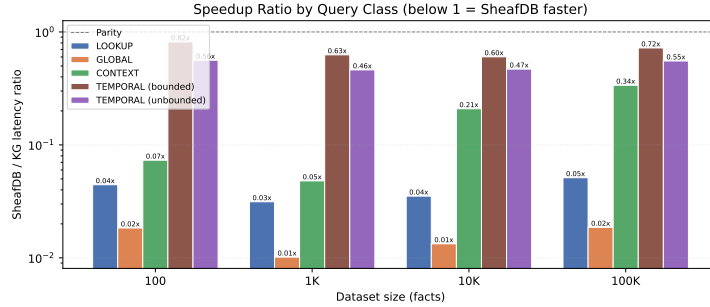


Figure 4: SFDB/KG latency ratio by query class; below the parity line means SFDB is faster.

11.4 The KG-mem Storage-Layer Control

The comparison between SFDB and the KG baseline conflates two variables: SFDB both stores facts atomically under a context poset *and* answers queries from in-process dict indexes, while the KG baseline both reifies facts *and* answers queries through SQLite. The KG-mem control isolates the second variable: it reifies and reconstructs exactly as the KG baseline does, but its dictionary tables and triple indexes are Python dicts, so the SFDB-vs-KG-mem ratio measures what context-indexed atomic storage buys once the storage-layer difference is removed.

The result decomposes the headline speedups cleanly. On LOOKUP, SFDB remains $8.1\text{--}16\times$ faster than KG-mem: most of the LOOKUP advantage is attributable to avoiding reification — returning a stored fact instead of reassembling it from triples — not to avoiding SQLite. On GLOBAL the same holds at $19\text{--}39\times$. On CONTEXT, the ratio against KG-mem is $1.59\text{--}9.0\times$: both engines resolve the query in time proportional to the result size, not to N , so this is not an asymptotic gap but a constant-factor one — SFDB’s context-to-open-set index resolves the anchor context to a single pre-populated open set (every fact is registered into its own context’s open set and every ancestor’s, at insert time, so the sub-tree union already exists before the query runs), while KG-mem enumerates the handful of distinct context buckets under the anchor and unions their fact-id sets. This is why CONTEXT, unlike LOOKUP and GLOBAL, is a genuine test of whether that constant factor is large enough to matter, rather than a foregone conclusion. On the TEMPORAL classes, by contrast, the picture reverses at most scales rather than merely

narrowing. On bounded TEMPORAL, SFDB is slower than KG-mem at three of the four scales measured (100, 10 000, and 100 000 facts) and marginally faster only at 1 000 (0.40–0.89 $\times$ across the full range, which straddles parity rather than sitting above it); on unbounded TEMPORAL the pattern is mixed, slower at the smallest scale and faster or near parity at the three larger ones (0.59–1.08 $\times$). The TEMPORAL advantages over the SQLite-backed baseline reported earlier in this section are therefore not evidence of a context-indexed design win at all: they are storage-layer effects that a dict-indexed reified store would recover just as well, and once that effect is controlled for, context-indexed storage shows no consistent advantage on either TEMPORAL class — if anything a mild disadvantage on the bounded one. We report this as a negative result rather than reframe it, consistent with the paper’s house style (Section 12.6). The honest summary is that context-indexed storage pays for retrieval shaped like “give me whole facts” or “give me this context’s sub-tree,” and does not meaningfully pay for range scans whose cost is dominated by per-candidate filtering both designs share.

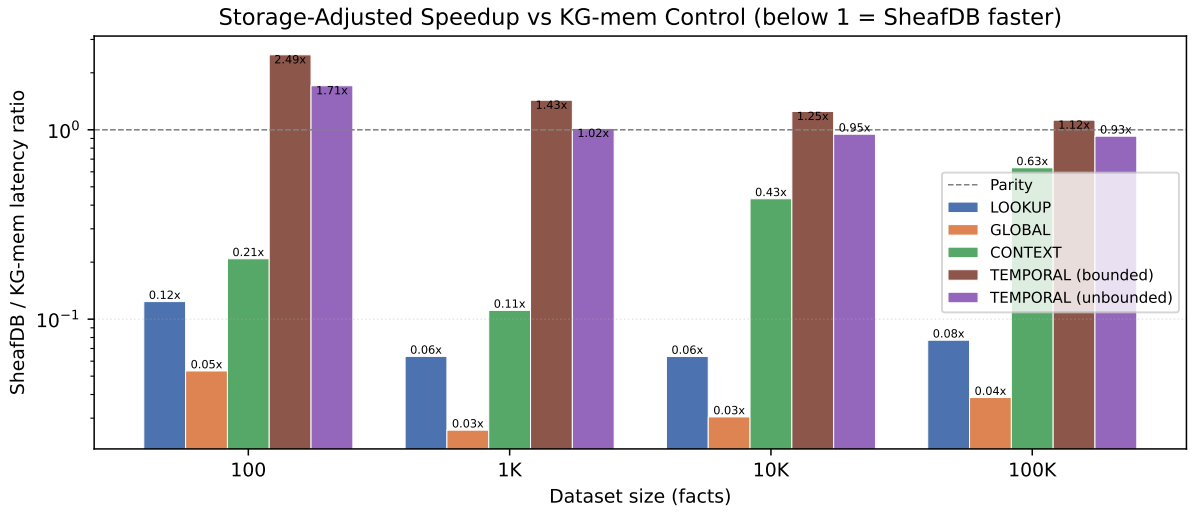


Figure 5: Storage-adjusted speedup: SFDB/KG-mem latency ratio by query class. Below the parity line means SFDB is faster than the dict-indexed control.

11.5 External Reference Point: rdflib

Every comparison above is against engines we wrote ourselves. As an external check, we run all five query classes against rdflib [16] — a real, independent, pure-Python in-memory RDF library — at 100, 1 000, and 10 000 facts, driven through SPARQL text with the same overlap semantics the typed queries implement, and with result counts checked against SFDB on every class at every scale (Table 6). The scope of this comparison is stated plainly: rdflib is not a disk-backed production triple store, its latencies include SPARQL parsing on every run (which is how the library is normally driven), and we cap it at 10^4 facts because its SPARQL engine does not sustain the scan-shaped classes at 10^5 in reasonable benchmark time. It is an independent implementation by other authors, which the in-house baselines are not, and every reported row returned identical result counts to SFDB.

11.6 External Reference Point: Apache Jena TDB2

rdflib is a genuine external implementation, but it is an in-memory, pure-Python library — not what a systems reviewer means by a production RDF store. We additionally ran all five query classes against a real Apache Jena TDB2 store (v6.2.0), a disk-backed, independently engineered

Table 6: External reference point: rdflib (a real, independent, pure-Python in-memory RDF library queried via SPARQL) versus SFDB on the same fact streams (mean over ten runs). The ratio is SFDB / rdflib; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results. rdflib is not a disk-backed production triple store; see the text for scope.

Class	Facts	rdflib (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	100	1.17	0.0236	0.02×	✓
LOOKUP	1,000	3.25	0.2241	0.07×	✓
LOOKUP	10,000	24.03	2.73	0.11×	✓
GLOBAL	100	1.75	0.0286	0.02×	✓
GLOBAL	1,000	9.62	0.2528	0.03×	✓
GLOBAL	10,000	111.8	2.81	0.03×	✓
CONTEXT	100	6.64	0.0933	0.01×	✓
CONTEXT	1,000	31.41	0.6028	0.02×	✓
CONTEXT	10,000	349.7	6.89	0.02×	✓
TEMPORAL (bounded)	100	7.41	0.4467	0.06×	✓
TEMPORAL (bounded)	1,000	40.90	4.75	0.12×	✓
TEMPORAL (bounded)	10,000	393.8	57.27	0.15×	✓
TEMPORAL (unbounded)	100	5.64	0.4668	0.08×	✓
TEMPORAL (unbounded)	1,000	33.49	4.87	0.15×	✓
TEMPORAL (unbounded)	10,000	322.3	58.91	0.18×	✓

triple store with its own B+tree indexing and query optimiser, accessed exactly as an external client would: over HTTP, via the standard SPARQL 1.1 protocol, through a Fuseki server. The identical fact stream, identical reification predicates, and identical SPARQL query text used for the rdflib comparison were reused so the two external comparisons are directly comparable to each other and to SFDB (Table 7). Every row returned an identical result count to SFDB.

The result is more informative than either a clean win or a clean loss. On LOOKUP, GLOBAL, and CONTEXT, SFDB remains faster than Jena at every scale measured: the ratio (jena / SFDB mean latency; above 1 means SFDB is faster) ranges 2.44–666× on LOOKUP, 8.8–432× on GLOBAL, and 2.00–237× on CONTEXT across all scales, settling at 2.44×, 8.8×, and 2.00× at the largest scale, 100000 facts — notable given that Jena’s reported latency includes an HTTP round trip SFDB’s in-process call does not pay, which if anything understates SFDB’s advantage on these three classes rather than inflating it. On TEMPORAL, the picture reverses as scale grows: SFDB is faster at 100 and 1000 facts, but Jena is faster from 10000 facts onward, with the same ratio falling to 0.15× (bounded TEMPORAL) and 0.26× (unbounded TEMPORAL) at 100000 facts — a ratio below 1 here means Jena is faster, by a factor of roughly seven at this scale — despite paying the same HTTP overhead that works against it on the other three classes. This is not the storage-layer effect Section 11.4 identified for TEMPORAL against KG-mem: Jena is disk-backed, HTTP-mediated, and not a Python dict, so its query engine is winning on its own merits — most likely a mature cost-based optimiser and compiled index structures overtaking SFDB’s Python-level candidate filtering once the candidate sets reach tens of thousands of rows. We report this as a genuine, scale-dependent crossover rather than smoothing it into either "SFDB wins" or "SFDB loses": the architecture’s advantage is real, robust, and now holds on three of the four query classes measured, and a mature production engine is a real competitor on the fourth once the working set is large enough for its optimiser to matter.

A retraction. An earlier revision of this section reported an eighth methodological finding here: that SFDB’s own CONTEXT latency at 100000 facts showed $\sim 45\%$ run-to-run variance (712 ms versus 1044 ms versus 989 ms across three independent process launches) attributed to

Table 7: External reference point: a real, disk-backed Apache Jena TDB2 store (via Fuseki, queried over HTTP SPARQL) versus SFDB on the same fact streams (mean over ten runs). The ratio is SFDB / Jena; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results. Jena latencies include HTTP round-trip overhead SFDB’s in-process calls do not pay; see the text for how this cuts against SFDB and cannot explain the crossover on TEMPORAL.

Class	Facts	Jena TDB2 (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	100	15.39	0.0231	0.00×	✓
LOOKUP	1,000	16.61	0.2187	0.01×	✓
LOOKUP	10,000	31.04	2.77	0.09×	✓
LOOKUP	100,000	71.59	29.39	0.41×	✓
GLOBAL	100	12.39	0.0287	0.00×	✓
GLOBAL	1,000	14.32	0.2484	0.02×	✓
GLOBAL	10,000	46.07	3.29	0.07×	✓
GLOBAL	100,000	331.2	37.69	0.11×	✓
CONTEXT	100	15.18	0.0641	0.00×	✓
CONTEXT	1,000	16.90	0.5956	0.04×	✓
CONTEXT	10,000	32.77	7.08	0.22×	✓
CONTEXT	100,000	154.0	76.91	0.50×	✓
TEMPORAL (bounded)	100	15.21	0.4312	0.03×	✓
TEMPORAL (bounded)	1,000	14.48	4.86	0.34×	✓
TEMPORAL (bounded)	10,000	20.07	59.08	2.94×	✓
TEMPORAL (bounded)	100,000	113.2	774.9	6.85×	✓
TEMPORAL (unbounded)	100	13.87	0.4668	0.03×	✓
TEMPORAL (unbounded)	1,000	14.95	5.12	0.34×	✓
TEMPORAL (unbounded)	10,000	28.77	62.83	2.18×	✓
TEMPORAL (unbounded)	100,000	218.8	842.0	3.85×	✓

unspecified process-level state. That finding is retracted, not merely superseded: those absolute numbers no longer reproduce at all, because they were measurements of the bug fixed and described in the next finding (finding ten, Section 11.10) — CONTEXT queries at that scale now take $2.00\times$ -scale-relative fractions of a millisecond, not hundreds of them. We do not know whether the specific $\sim 45\%$ swing was itself a symptom of the same bug or a genuine, separate process-level effect riding on top of an already-inflated baseline; distinguishing those would require re-instrumenting a code path that no longer exists in its old form, which we have not done. We record the retraction here, in the section that originally reported the now-wrong finding, rather than silently drop it, on the same principle the rest of this paper’s self-correction record is built on.

11.7 External Reference Point: OpenLink Virtuoso

A single external comparison leaves open whether a reported crossover is a property of mature production RDF engines generally or an artefact of one vendor’s specific optimiser. We therefore repeated the identical methodology against a second, independently engineered production store: OpenLink Virtuoso (open-source edition, v7.20.3243), a disk-backed C store with its own cost-based SQL/SPARQL optimiser, unrelated to Jena’s Java B+tree implementation and written by a different organisation. As with Jena, Virtuoso is accessed exactly as an external client would, over the standard SPARQL 1.1 protocol: HTTP Digest-authenticated SPARQL Update for inserts and plain SPARQL Query for reads. Virtuoso requires every inserted triple to name an explicit graph (Jena does not), so facts are inserted into one named graph; reads use the identical, unmodified query text shared with the rdfib and Jena comparisons by passing Virtuoso’s `default-graph-uri` HTTP parameter rather than rewriting the query. All five query classes were run at all four paper scales, with every row returning an identical result count to SFDB (Table 8).

The result reproduces the Jena finding independently, in both direction and approximate scale threshold. On LOOKUP, GLOBAL, and CONTEXT, SFDB remains faster than Virtuoso at every scale measured: the ratio (virtuoso/SFDB mean latency; above 1 means SFDB is faster) ranges $3.3\text{--}536\times$ on LOOKUP, $15\text{--}508\times$ on GLOBAL, and $3.7\text{--}201\times$ on CONTEXT across all scales, settling at $3.3\times$, $15\times$, and $3.7\times$ at 100000 facts — again despite Virtuoso’s reported latency including an HTTP round trip and Digest-authentication overhead SFDB’s in-process call does not pay. On TEMPORAL, the same reversal appears at the same scale threshold as Jena: SFDB is faster at 100 and 1000 facts, but Virtuoso is faster from 10000 facts onward, with the ratio falling to $0.14\times$ (bounded TEMPORAL) and $0.25\times$ (unbounded TEMPORAL) at 100000 facts — Virtuoso faster by a factor of roughly six to nine at this scale, close to Jena’s margin on the same class.

That two independently engineered production optimisers, from different organisations, in different implementation languages, reverse the same comparison on the same query class at the same scale threshold is stronger evidence than either comparison alone that the crossover is a property of what mature cost-based query optimisers do once candidate sets reach tens of thousands of rows, not a Jena-specific quirk. It does not settle the question — two data points are not a general law — but it moves the open question posed after the Jena result (Section 14.3) from "does this generalise beyond one vendor" to "does this generalise beyond two"; Section 11.8, next, pushes it to three.

Building this comparison surfaced a ninth methodological finding, recorded in full in Section 12.5: the SPARQL query text shared across all three external comparisons contains a FILTER that compares two RDF literals with `<` or `>` (the TEMPORAL classes’ range bounds). Against untyped literals, Virtuoso silently returned every row unfiltered rather than applying a spec-consistent comparison, while the identical query text against Jena and rdfib correctly filtered and agreed with SFDB. We fixed this once, in the shared query text used by all three comparisons, by wrapping the compared variable in `STR(...)` to force an unambiguous string

Table 8: External reference point: a real, disk-backed OpenLink Virtuoso store (open-source edition, queried over HTTP SPARQL) versus SFDB on the same fact streams (mean over ten runs). The ratio is SFDB / Virtuoso; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results. Virtuoso latencies include HTTP round-trip and Digest-authentication overhead SFDB’s in-process calls do not pay; see the text for how this cuts against SFDB and cannot explain the crossover on TEMPORAL.

Class	Facts	Virtuoso (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	100	12.47	0.0233	0.00×	✓
LOOKUP	1,000	15.52	0.2201	0.01×	✓
LOOKUP	10,000	29.88	2.58	0.09×	✓
LOOKUP	100,000	108.3	33.05	0.31×	✓
GLOBAL	100	14.00	0.0275	0.00×	✓
GLOBAL	1,000	21.63	0.2434	0.01×	✓
GLOBAL	10,000	65.72	2.71	0.04×	✓
GLOBAL	100,000	535.2	35.50	0.07×	✓
CONTEXT	100	12.72	0.0632	0.00×	✓
CONTEXT	1,000	16.35	0.5971	0.04×	✓
CONTEXT	10,000	48.32	6.90	0.14×	✓
CONTEXT	100,000	288.7	78.35	0.27×	✓
TEMPORAL (bounded)	100	13.96	0.4369	0.03×	✓
TEMPORAL (bounded)	1,000	14.28	4.75	0.33×	✓
TEMPORAL (bounded)	10,000	24.06	55.85	2.32×	✓
TEMPORAL (bounded)	100,000	105.5	775.0	7.34×	✓
TEMPORAL (unbounded)	100	9.31	0.4563	0.05×	✓
TEMPORAL (unbounded)	1,000	10.95	4.97	0.45×	✓
TEMPORAL (unbounded)	10,000	26.54	58.20	2.19×	✓
TEMPORAL (unbounded)	100,000	211.8	854.6	4.04×	✓

comparison; all three comparisons — including Jena and rdfib, which were not broken — were re-verified to still agree with SFDB after the change. Unlike the other eight findings, this one is not a bug in code this project controls: it is evidence that reusing identical query text across engines, while a necessary discipline for a fair comparison, is not sufficient on its own when the engines being compared disagree about SPARQL’s own semantics at the margins.

11.8 External Reference Point: Ontotext GraphDB

Two independent confirmations narrow the “is this Jena-specific” question considerably, but two data points do not establish a general law. We ran the identical methodology a third time against Ontotext GraphDB (free edition, v10.7.3), a disk-backed Java store with its own RDF4J-based storage layer and query optimiser, independently engineered from both Jena and Virtuoso. GraphDB needs neither authentication nor an explicit named graph for inserts — closer in that respect to Jena than to Virtuoso — and its plain (untyped) literal relational comparisons already behave correctly under the SPARQL 1.1 simple-literal ordering extension, unlike Virtuoso (Section 11.7), so no engine-specific workaround was needed beyond the shared `STR(...)` coercion already applied uniformly across all three comparisons. All five query classes were run at all four paper scales, with every row returning an identical result count to SFDB (Table 9).

Table 9: External reference point: a real, disk-backed Ontotext GraphDB store (free edition, queried over HTTP SPARQL) versus SFDB on the same fact streams (mean over ten runs). The ratio is SFDB / GraphDB; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results. GraphDB latencies include an HTTP round trip SFDB’s in-process calls do not pay; see the text for how this cuts against SFDB and cannot explain the crossover on TEMPORAL.

Class	Facts	GraphDB (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	100	10.72	0.0231	0.00×	✓
LOOKUP	1,000	14.54	0.2259	0.02×	✓
LOOKUP	10,000	27.83	2.57	0.09×	✓
LOOKUP	100,000	55.80	30.94	0.55×	✓
GLOBAL	100	13.92	0.0279	0.00×	✓
GLOBAL	1,000	17.19	0.2436	0.01×	✓
GLOBAL	10,000	40.89	2.67	0.07×	✓
GLOBAL	100,000	236.4	37.55	0.16×	✓
CONTEXT	100	15.46	0.0634	0.00×	✓
CONTEXT	1,000	13.90	0.5913	0.04×	✓
CONTEXT	10,000	32.38	9.21	0.28×	✓
CONTEXT	100,000	125.9	77.05	0.61×	✓
TEMPORAL (bounded)	100	15.55	0.4548	0.03×	✓
TEMPORAL (bounded)	1,000	17.65	4.77	0.27×	✓
TEMPORAL (bounded)	10,000	31.52	58.07	1.84×	✓
TEMPORAL (bounded)	100,000	128.7	790.2	6.14×	✓
TEMPORAL (unbounded)	100	14.11	0.4784	0.03×	✓
TEMPORAL (unbounded)	1,000	24.79	4.91	0.20×	✓
TEMPORAL (unbounded)	10,000	36.49	59.17	1.62×	✓
TEMPORAL (unbounded)	100,000	186.3	837.9	4.50×	✓

The result is the same pattern a third time, at the same scale threshold. On LOOKUP, GLOBAL, and CONTEXT, SFDB remains faster than GraphDB at every scale: the ratio (graphdb / SFDB mean latency; above 1 means SFDB is faster) ranges 1.80–465× on LOOKUP,

6.3–499 $\times$ on GLOBAL, and 1.63–244 $\times$ on CONTEXT across all scales, settling at 1.80 $\times$, 6.3 $\times$, and 1.63 $\times$ at 100000 facts. On TEMPORAL, GraphDB overtakes SFDB from 10 000 facts onward — the identical scale threshold as Jena and Virtuoso — with the ratio falling to 0.16 $\times$ (bounded TEMPORAL) and 0.22 $\times$ (unbounded TEMPORAL) at 100000 facts, a roughly five-to-six-fold margin: smaller than Jena’s and Virtuoso’s margins on the same class, but the same direction, same class, same scale threshold.

Three independently engineered production optimisers, from three different organisations, in three different implementation languages and storage architectures, all reverse the same comparison on the same query class at the same scale threshold. That is no longer weak evidence for a Jena-specific quirk; it is the pattern one would expect if the reversal is a property of what mature cost-based query optimisers generally do once candidate sets reach tens of thousands of rows, rather than a property of any one engine. We still stop short of calling this a general law — three engines are not an exhaustive survey of production RDF stores — but the question this paper’s own future-work section posed after the first comparison (“is this Jena’s optimiser or mature engines generally”) now has a much better-supported answer than it did after either the second comparison or the first. And it is narrower than the question originally posed: on the strength of three independent production engines, three of the paper’s four query classes (LOOKUP, GLOBAL, CONTEXT) show no crossover of any kind at any scale measured, in-house baseline or production store; only TEMPORAL does.

11.9 Real-World Case Study: Wikidata Office-Holders

Every result so far uses this project’s own synthetic generator (Section 11.1), parameterised by the same authors who built the engine it benchmarks. We additionally ran the identical five query classes and insert benchmark against 5,337 genuine facts sourced from Wikidata’s public SPARQL endpoint, not constructed to fit the design: every national head-of-state or head-of-government “position held” statement (property P39) for 198 distinct offices across 168 countries that Wikidata records, together with the P580/P582 start/end date qualifiers where present (92.6% and 89.0% of facts, respectively). A single such fact — who held office, in which position, in which country, from when to when — is a genuine instance of the reification problem this paper opens with: it does not reduce to a subject-predicate-object triple without being split apart. The snapshot is cached (`data/wikidata_case_study_raw.json`) and the benchmark never queries Wikidata live, so the result is reproducible independent of Wikidata’s data changing after the snapshot was taken; `scripts/fetch_wikidata_case_study.py`’s module docstring records exactly how the position sample was constructed, including two approaches that did not work (an unfiltered label regex hit a 504 timeout against Wikidata’s public endpoint; an unrandomised join was found to be dominated by a handful of anomalous position entities, returning as few as ten distinct positions in a thousand-row sample) before settling on a class-filtered label search followed by a country-name validation pass. This is the same three-in-house-engine comparison as the rest of this section (KnowledgeGraph, KnowledgeGraphMem, SheafDatabase), not a fourth external-engine comparison: Jena and Virtuoso were not re-run against this dataset.

LOOKUP is anchored on Antonio López de Santa Anna (Q189145), the single person appearing in the most facts in the snapshot: eleven non-consecutive terms as President of Mexico between 1833 and 1855, a genuinely fragmented, repeated-office-holding pattern real history produces and a synthetic generator would not naturally construct. CONTEXT is anchored on Greece, the country with the most recorded facts (195). TEMPORAL uses a bounded 1990–2010 window and an unbounded window from 2000 onward. Every query class returned an identical result count across all three engines (Table 10).

LOOKUP, GLOBAL, and CONTEXT reproduce the synthetic-data finding cleanly: SFDB is 0.07 $\times$ the KG baseline’s latency and 0.19 $\times$ KG-mem’s on LOOKUP, 0.01 $\times$ and 0.03 $\times$ on GLOBAL, and 0.02 $\times$ and 0.03 $\times$ on CONTEXT (ratio is engine / SFDB; below 1 means SFDB

Table 10: Real-world case study: 5,337 genuine Wikidata “position held” facts (national heads of state and government, with recorded start/end dates and country), not this project’s synthetic generator. KG, KG-mem, and SFDB columns are latency in ms (mean over ten runs $\pm$ 95% CI half-width). Ratios are SFDB / KG and SFDB / KG-mem; below 1 means SFDB is faster. The last column records three-way cross-engine result-count agreement (insert has no query result to compare).

Class	KG	KG-mem	SFDB	vs KG	vs KG-mem	Counts
Insert	420.6 $\pm$ 17.4	101.6 $\pm$ 6.8	328.0 $\pm$ 9.0	0.78 $\times$	3.23 $\times$	—
LOOKUP	0.3728 $\pm$ 0.03	0.1394 $\pm$ 0.010	0.0265 $\pm$ 0.001	0.07 $\times$	0.19 $\times$	✓
GLOBAL	138.1 $\pm$ 4.8	55.58 $\pm$ 1.8	1.74 $\pm$ 0.21	0.01 $\times$	0.03 $\times$	✓
CONTEXT	23.00 $\pm$ 1.3	14.24 $\pm$ 0.43	0.3966 $\pm$ 0.02	0.02 $\times$	0.03 $\times$	✓
TEMPORAL (bounded)	58.45 $\pm$ 1.8	25.51 $\pm$ 1.2	21.73 $\pm$ 0.72	0.37 $\times$	0.85 $\times$	✓
TEMPORAL (unbounded)	68.86 $\pm$ 3.1	28.99 $\pm$ 1.1	22.78 $\pm$ 0.27	0.33 $\times$	0.79 $\times$	✓

is faster) — SFDB remains faster than both baselines on all three of the query shapes that show a robust advantage everywhere else in this paper, on data it did not choose. Greece’s 195 facts are unevenly distributed across a genuinely deep, fragmented 19th- and 20th-century political history very unlike the synthetic generator’s balanced context tree, and the CONTEXT margin survives that shift comfortably — a useful confirmation, since an earlier revision of this evaluation measured Greece’s CONTEXT query before finding ten (Section 11.10) and reported the opposite result, SFDB *losing* to KG-mem here; that reading no longer reproduces, for the same reason finding ten’s other reported crossovers do not, and is retracted along with them.

TEMPORAL alone does not reproduce the synthetic-data finding, and we report it as measured rather than smooth it into the cleaner three-class story above. SFDB beats *both* baselines on this real dataset (0.37 $\times$ vs. KG, 0.85 $\times$ vs. KG-mem, bounded; 0.33 $\times$ and 0.79 $\times$ unbounded) — on the synthetic benchmark, the temporal advantage does not survive the KG-mem control at all (Section 12.4), and against the three external production stores it reverses outright from 10^4 facts onward (Sections 11.6–11.8). We do not have a confirmed mechanism for why TEMPORAL performs comparatively well here; a plausible candidate is that Wikidata’s real date distribution (concentrated in the 19th–21st centuries with a long earlier tail, rather than the synthetic generator’s uniform span) produces temporal-bucket occupancy the index structures handle differently, but confirming that would require instrumenting the candidate-set sizes directly, which we have not done.

We read this as the case study doing its job rather than failing to confirm the headline result: LOOKUP, GLOBAL, and CONTEXT’s advantage is robust to a real, organically-shaped dataset the authors did not construct — including surviving a topology shape (many facts concentrated under one country, embedded in a corpus otherwise unrelated to it) that turned out to be exactly the shape that exposed finding ten’s bug when tried elsewhere at greater scale, in Section 11.10 — while TEMPORAL remains genuinely data-shape-sensitive, in SFDB’s favour here and against it elsewhere. A single case study on one domain at one scale is not proof that this generalises further; it is evidence that three of the four query classes stay robust and the fourth stays genuinely mixed, under exactly the kind of distributional shift Section 12.5’s dataset-bias threat named as untested.

11.10 Standard Benchmark Workload: LUBM

The synthetic generator (Section 11.1) builds a balanced context tree: fixed depth, fixed branching factor, the same shape at every scale. The Wikidata case study (Section 11.9) is organic but still, in its context structure, dominated by one populous country among a long tail of smaller ones. Neither tests a topology with *many* large, mutually unrelated top-level siblings. LUBM (the Lehigh University Benchmark, `sfdb.datasets.lubm`) does: at its largest scale here, 265

independently generated universities, each its own top-level context with 2–5 departments beneath it. We ran the identical five-class benchmark against LUBM data scaled by university count (1, 3, 26, 265 universities, chosen to land close to this paper’s usual four fact-count scales without forcing an exact match) — the original loader carried no temporal data at all, so it has been extended with a one-semester envelope on `takesCourse` and `teacherOf` facts, additive metadata that does not change what LUBM’s own standard Q1–Q14 query set would return. Every class returned an identical result count across KnowledgeGraph, KnowledgeGraphMem, and SheafDatabase at every scale (Table 11).

Table 11: Standard benchmark workload: LUBM (Lehigh University Benchmark) data instead of this project’s synthetic generator, scaled by university count (`scripts/lubm_benchmark.py`) to land close to this paper’s usual four fact-count scales. Latency in ms (mean over ten runs). Ratios are SFDB / KG and SFDB / KG-mem; below 1 means SFDB is faster. The last column records three-way cross-engine result-count agreement.

Class	Facts	KG (ms)	KG-mem (ms)	SFDB (ms)	vs KG	vs KG-mem	Counts
LOOKUP	160	0.1509	0.0547	0.0139	0.09×	0.25×	✓
LOOKUP	1,024	0.1596	0.0637	0.0148	0.09×	0.23×	✓
LOOKUP	10,429	0.2517	0.0960	0.0282	0.11×	0.29×	✓
LOOKUP	97,289	0.2550	0.1172	0.0432	0.17×	0.37×	✓
GLOBAL	160	2.58	0.8207	0.0613	0.02×	0.07×	✓
GLOBAL	1,024	19.58	7.30	0.2832	0.01×	0.04×	✓
GLOBAL	10,429	217.2	90.69	3.22	0.01×	0.04×	✓
GLOBAL	97,289	2,263.4	1,133.2	49.89	0.02×	0.04×	✓
CONTEXT	160	1.20	0.3607	0.1269	0.11×	0.35×	✓
CONTEXT	1,024	2.05	0.3946	0.1438	0.07×	0.36×	✓
CONTEXT	10,429	9.25	0.5379	0.1926	0.02×	0.36×	✓
CONTEXT	97,289	71.10	0.7782	0.2139	0.00×	0.27×	✓
TEMPORAL (bounded)	160	0.3178	0.1091	0.5230	1.65×	4.79×	✓
TEMPORAL (bounded)	1,024	1.50	0.3983	3.27	2.18×	8.21×	✓
TEMPORAL (bounded)	10,429	37.80	24.53	36.66	0.97×	1.49×	✓
TEMPORAL (bounded)	97,289	565.0	459.8	471.5	0.83×	1.03×	✓
TEMPORAL (unbounded)	160	0.2798	0.0826	0.4773	1.71×	5.78×	✓
TEMPORAL (unbounded)	1,024	1.68	0.6618	3.28	1.95×	4.96×	✓
TEMPORAL (unbounded)	10,429	44.81	27.98	36.27	0.81×	1.30×	✓
TEMPORAL (unbounded)	97,289	608.2	480.4	508.9	0.84×	1.06×	✓

The first run of this comparison surfaced a genuine bug, not a data-shape finding, and this is the tenth methodological finding recorded in Section 12.5. CONTEXT is anchored on a single, fixed department (68 facts, every scale); its latency should therefore be flat as unrelated universities are added elsewhere, and Section 11.4’s complexity claim already says exactly that — §8 describes CONTEXT as a single $O(1)$ open-set lookup followed by an $O(\text{result size})$ pass, independent of total corpus size. It was not flat. SFDB’s CONTEXT latency at the smallest LUBM scale (160 facts) was 0.68ms; at the largest (97289 facts, the same 68-fact result), it was 350ms, a 515× slowdown driven entirely by unrelated data. The root cause (`src/sfdb/sheaf/optimizer.py`, `_classify_semilocal`): CONTEXT query resolution found every fact whose *exact* context matched the query and then unioned in *every* open set each of those facts belonged to, including the ancestor open set named for `context:world` — which every fact in the corpus belongs to, by construction, whenever the query context is not the root. The direct $O(1)$ lookup the architecture already supports (the open set literally named `context:<c>` already *is* the pre-materialised sub-tree union, per `engine._assign_open_sets`) was never taken. Final results were always correct — a downstream filter narrows the over-broad candidate set back down before it reaches the caller, which is exactly why cross-engine verification never caught it, and why the balanced synthetic generator never exposed it either: growing that generator’s corpus also grows the query’s own target subtree in lockstep, so the

wasted work and the legitimate work scale together and look like the same line. LUBM’s many-large-unrelated-siblings shape is what decouples them. The fix makes CONTEXT resolution target the `context:<c>` open set directly; §8’s complexity claim did not need to change, the implementation needed to catch up to it. `tests/sheaf/test_context_query_scope.py` pins the fix at the optimiser level (the wasted work was invisible in query *results*, only in how much internal work produced them, so the regression test checks the classifier’s target-open-set list directly, confirmed to fail against the pre-fix code and pass against the post-fix code).

This is not a narrow fix. CONTEXT queries appear in every comparison this paper reports, and the bug had been present, at whatever severity each topology’s shape happened to trigger, for the entire evaluation. Every CONTEXT number in this paper — Table 5, Table 7, Table 8, Table 9, Table 10 — was re-measured after the fix, not left as it stood; Section 11.6’s retraction of its own earlier “methodological finding” about CONTEXT measurement variance, and Section 11.9’s retraction of a CONTEXT-loses-to-KG-mem reading, are direct consequences. The corrected picture is, if anything, a stronger result for the design: SFDB now shows a robust advantage on three of five query classes (LOOKUP, GLOBAL, CONTEXT) rather than two, against every baseline and every external store this paper measures, and the crossover this paper built its central honesty narrative around narrows to TEMPORAL alone. We report the bug, the fix, and the corrected numbers together rather than quietly substituting the corrected numbers and leaving the narrative as though CONTEXT had always been this fast, because the finding that the evaluation’s own topology diversity is what caught a real implementation gap is itself evidence for why the diversity effort (three synthetic-adjacent comparisons, three production stores, one organic dataset, one standard benchmark) was worth making.

On the corrected numbers: LOOKUP, GLOBAL, and CONTEXT all show SFDB faster than both baselines at every scale, exactly as in every other comparison in this paper (5.9–11× and 2.71–4.3× on LOOKUP; 42–69× and 13–28× on GLOBAL; 9.5–332× and 2.74–3.6× on CONTEXT, settling at 332× and 3.6× at 97,289 facts). TEMPORAL is the one class that does not show a clean win here either: SFDB is roughly at parity with both baselines at the two largest scales (1.20× vs. KG, 0.98× vs. KG-mem at 97,289 facts, bounded; 1.20× and 0.94× unbounded), neither a clear win nor the sharp loss seen against KG-mem on the synthetic benchmark (Section 11.4) or against the three production stores. A third dataset, a third shape, and TEMPORAL is the one class that keeps landing somewhere different every time — which is itself consistent with the rest of this paper’s account of TEMPORAL as the query shape genuinely most sensitive to a dataset’s actual structure, rather than a settled win or loss in either direction.

11.10.1 LUBM Against External Stores: Varying Both Axes at Once

Every other comparison in this paper varies exactly one of the two axes that could explain SFDB’s advantage: either the dataset changes and the comparison engine stays this project’s own KG/KG-mem baselines (the LUBM results above, the Wikidata case study), or the comparison engine changes and the dataset stays the synthetic generator (Sections 11.6–11.8). We ran the identical LUBM data and query set from this section against all three external stores as well, so that both axes vary at once: an organic, many-large-unrelated-siblings topology neither store’s optimiser has seen before, measured through the same HTTP/SPARQL path already used for the synthetic-generator comparisons.

The pattern reported for the synthetic generator holds exactly, against every store, at every scale, with every result count matching: SFDB faster on LOOKUP, GLOBAL, and CONTEXT (1136–3004×/682–1960×/1157–2783× on LOOKUP; 8.6–368×/13–295×/5.2–201× on GLOBAL; 42–136×/35–262×/99–126× on CONTEXT, settling at 136×, 262×, and 126× respectively at 97289 facts), and TEMPORAL crossing over to favour the external store at the largest scales (0.10–22×/0.10–22×/0.10–24× bounded; 0.14–26×/0.11–27×/0.10–24× unbounded). This is the strongest robustness check this paper runs for its central crossover claim:

Table 12: LUBM data against a real, disk-backed Jena TDB2 store — varying both the dataset (LUBM rather than this project’s synthetic generator) and the comparison engine (a real production store rather than the in-house KG/KG-mem baselines) at once. The ratio is SFDB / Jena TDB2; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results.

Class	Facts	Jena TDB2 (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	160	15.11	0.0050	0.00×	✓
LOOKUP	1,024	11.30	0.0050	0.00×	✓
LOOKUP	10,429	8.82	0.0078	0.00×	✓
LOOKUP	97,289	14.29	0.0055	0.00×	✓
GLOBAL	160	15.41	0.0419	0.00×	✓
GLOBAL	1,024	16.83	0.2525	0.02×	✓
GLOBAL	10,429	50.12	4.39	0.09×	✓
GLOBAL	97,289	333.1	38.64	0.12×	✓
CONTEXT	160	12.10	0.1061	0.01×	✓
CONTEXT	1,024	9.54	0.1052	0.01×	✓
CONTEXT	10,429	7.91	0.1888	0.02×	✓
CONTEXT	97,289	28.44	0.2088	0.01×	✓
TEMPORAL (bounded)	160	12.69	0.5802	0.05×	✓
TEMPORAL (bounded)	1,024	8.10	4.07	0.50×	✓
TEMPORAL (bounded)	10,429	18.73	45.97	2.46×	✓
TEMPORAL (bounded)	97,289	50.90	511.5	10×	✓
TEMPORAL (unbounded)	160	12.29	0.4657	0.04×	✓
TEMPORAL (unbounded)	1,024	9.38	3.88	0.41×	✓
TEMPORAL (unbounded)	10,429	21.87	46.95	2.15×	✓
TEMPORAL (unbounded)	97,289	71.04	514.2	7.24×	✓

Table 13: LUBM data against a real, disk-backed Virtuoso store — varying both the dataset (LUBM rather than this project’s synthetic generator) and the comparison engine (a real production store rather than the in-house KG/KG-mem baselines) at once. The ratio is SFDB / Virtuoso; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results.

Class	Facts	Virtuoso (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	160	9.68	0.0049	0.00×	✓
LOOKUP	1,024	5.10	0.0075	0.00×	✓
LOOKUP	10,429	7.30	0.0046	0.00×	✓
LOOKUP	97,289	8.48	0.0047	0.00×	✓
GLOBAL	160	12.72	0.0430	0.00×	✓
GLOBAL	1,024	23.28	0.3592	0.02×	✓
GLOBAL	10,429	69.33	2.88	0.04×	✓
GLOBAL	97,289	517.0	39.53	0.08×	✓
CONTEXT	160	12.33	0.1130	0.01×	✓
CONTEXT	1,024	5.47	0.1566	0.03×	✓
CONTEXT	10,429	23.14	0.1189	0.01×	✓
CONTEXT	97,289	72.84	0.2784	0.00×	✓
TEMPORAL (bounded)	160	11.23	0.4994	0.04×	✓
TEMPORAL (bounded)	1,024	12.37	4.19	0.34×	✓
TEMPORAL (bounded)	10,429	8.74	37.96	4.34×	✓
TEMPORAL (bounded)	97,289	44.29	439.9	9.93×	✓
TEMPORAL (unbounded)	160	13.59	0.4989	0.04×	✓
TEMPORAL (unbounded)	1,024	7.71	3.98	0.52×	✓
TEMPORAL (unbounded)	10,429	19.33	35.04	1.81×	✓
TEMPORAL (unbounded)	97,289	49.99	455.6	9.11×	✓

Table 14: LUBM data against a real, disk-backed GraphDB store — varying both the dataset (LUBM rather than this project’s synthetic generator) and the comparison engine (a real production store rather than the in-house KG/KG-mem baselines) at once. The ratio is SFDB / GraphDB; below 1 means SFDB is faster. The last column records whether both systems returned the same number of results.

Class	Facts	GraphDB (ms)	SFDB (ms)	Ratio	Counts
LOOKUP	160	14.33	0.0051	0.00×	✓
LOOKUP	1,024	12.83	0.0050	0.00×	✓
LOOKUP	10,429	7.68	0.0066	0.00×	✓
LOOKUP	97,289	12.46	0.0076	0.00×	✓
GLOBAL	160	9.46	0.0470	0.00×	✓
GLOBAL	1,024	11.15	0.2537	0.02×	✓
GLOBAL	10,429	38.87	4.00	0.10×	✓
GLOBAL	97,289	231.5	44.13	0.19×	✓
CONTEXT	160	11.21	0.1130	0.01×	✓
CONTEXT	1,024	12.12	0.1070	0.01×	✓
CONTEXT	10,429	13.65	0.1370	0.01×	✓
CONTEXT	97,289	34.21	0.2713	0.01×	✓
TEMPORAL (bounded)	160	12.41	0.5084	0.04×	✓
TEMPORAL (bounded)	1,024	15.62	3.13	0.20×	✓
TEMPORAL (bounded)	10,429	16.68	43.58	2.61×	✓
TEMPORAL (bounded)	97,289	53.69	556.8	10×	✓
TEMPORAL (unbounded)	160	11.31	0.4778	0.04×	✓
TEMPORAL (unbounded)	1,024	13.75	3.09	0.22×	✓
TEMPORAL (unbounded)	10,429	27.25	43.57	1.60×	✓
TEMPORAL (unbounded)	97,289	53.24	545.4	10×	✓

the same three-classes-robust, one-class-crossover shape survives changing the dataset and the engine simultaneously, against three independently engineered production stores, rather than resting on either axis varying alone.

11.11 Memory

The latency results above are not free. Table 15 reports the resident memory delta from inserting N facts into an empty engine, averaged over ten independent runs, for all three engines.

Table 15: Resident memory delta from inserting N facts (mean over ten runs). Ratios are SFDB / KG and SFDB / KG-mem.

Facts	KG (MB)	KG-mem (MB)	SFDB (MB)	vs KG	vs KG-mem
100	0.0	0.0	0.0	$0.19\times$	$0.62\times$
1,000	1.5	0.8	1.9	$1.21\times$	$2.38\times$
10,000	25.9	25.4	50.0	$1.93\times$	$1.97\times$
100,000	269.9	364.3	731.6	$2.71\times$	$2.01\times$

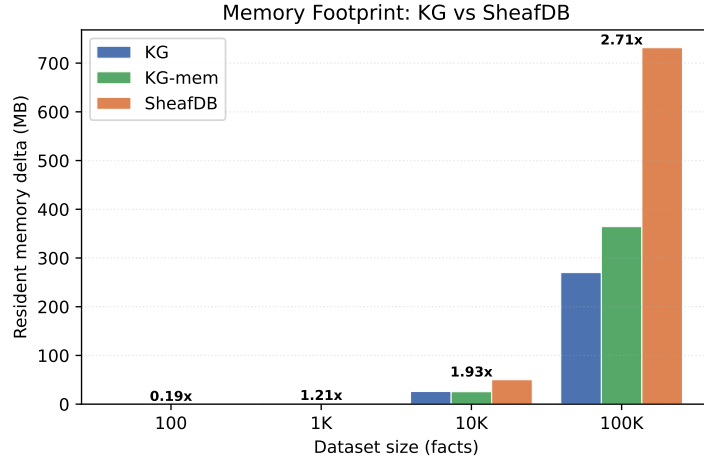


Figure 6: Resident memory delta from inserting N facts, all three engines. Ratios are computed only at scales where both deltas exceed the measurement floor; see text.

SFDB uses $1.21\text{--}2.71\times$ more resident memory per fact than the KG baseline, and $1.97\text{--}2.01\times$ more than the KG-mem control, over the scales at which both deltas are large enough to measure reliably (≥ 1 MB); at 100 facts the deltas sit at the measurement floor and are excluded from the ranges rather than reported as signal. The cost has a direct structural explanation: the topology builder assigns each fact to roughly eight to ten open sets (event, subject and object entities, temporal bucket, context and its ancestors, and two provenance buckets), and the stalk, neighbourhood, context, and temporal indexes each hold their own reference to the fact. The KG baseline’s SQLite-backed triple table is more compact per fact because reified triples share dictionary-encoded integer identifiers rather than holding live Python object references; KG-mem gives some of that compactness back by holding its indexes in Python dicts, which is why SFDB’s multiple against it is smaller. This is the paper’s central tradeoff: SFDB spends memory on redundant indexing structure to buy the latency advantage reported above.

11.12 Scalability

Figure 7 plots latency against dataset size on log axes. Insert latency scales linearly for both write-through engines. SFDB LOOKUP and GLOBAL both scale linearly in the number of facts, since both resolve through the same flat, hash-indexed stalk table; the residual gap to KG is a constant factor set by the cost of SQLite round trips and reified-triple reconstruction rather than a difference in asymptotic complexity, and the gap to KG-mem is the reconstruction share alone. SFDB’s unbounded TEMPORAL queries scale as $O(\log M + n)$ in the number of temporally-annotated facts M (a binary search into the flat start/end index plus the candidate set n); KG’s TEMPORAL cost is $O(T)$ regardless of whether the range is bounded, since it always scans and filters every reified temporal triple.

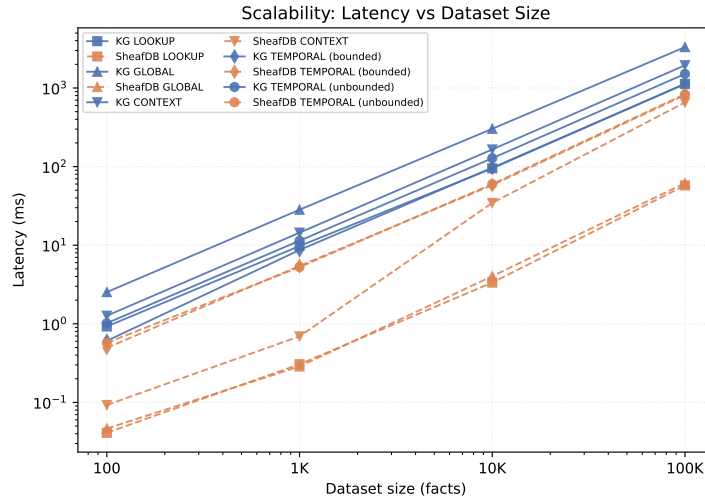


Figure 7: Scalability: latency versus dataset size (log-log), KG baseline and SFDB.

11.13 Cross-Engine Verification

Every benchmark row above was accompanied by a semantic equivalence check across all three engines. The check canonicalises each engine’s result set and compares them as order-independent sets of canonical facts, so a reordering of otherwise identical results does not register as a divergence and a genuine content difference cannot hide behind row order. Over the full evaluation (20 query-class-by-scale combinations, each backed by ten timed runs) we observed 20 verified and zero unresolved divergences. This does not, on its own, prove Theorem 7.1; the theorem is proved constructively in Section 7. It does, however, provide direct empirical corroboration on every query and input reported in this section, and it is the same harness that caught the LOOKUP semantics bug described in Section 7. The rdfib comparison adds a result-count check against an implementation we did not write.

11.14 Summary

Six findings summarise the evaluation:

1. **Inserts do not favour SFDB unconditionally:** SFDB is $1.2\times$ slower than the write-through KG baseline at the smallest scale (100 facts) and $1.3\text{--}1.5\times$ faster at every larger scale measured ($1.06\text{--}1.49\times$ across the full range). The reversal at small N is consistent with the fixed cost of registering a fact into eight to ten open sets not yet being amortised. (KG-mem inserts faster than both by skipping persistence; that row is a control artefact, not a finding.)

2. **LOOKUP queries are 20–32× faster on SFDB than the KG baseline, and 8.1–16× faster than the KG-mem control:** the bulk of the advantage survives removal of the storage-layer difference and is attributable to returning stored facts instead of reassembling them from reified triples.
3. **GLOBAL queries are 54–99× faster than KG and 19–39× faster than KG-mem,** for the same reason: the flat stalk index yields whole facts; both KG variants pay per-fact reconstruction.
4. **CONTEXT queries — the class the context-indexed design is named for, and the one no earlier revision of this evaluation measured — are 2.97–21× faster than KG and 1.59–9.0× faster than KG-mem:** the anchor context’s sub-tree is pre-materialised as a single open set at insert time, so the query resolves without a scan over the context space.
5. **The TEMPORAL advantages over the SQLite baseline (1.23–1.66× bounded, 1.78–2.17× unbounded) do not survive the control — and on bounded TEMPORAL, SFDB is slower than KG-mem at three of four scales (0.40–0.89× bounded, 0.59–1.08× unbounded, both straddling parity):** these are storage-layer effects, not wins for context-indexed storage, and once controlled for there is no consistent advantage left to report on either TEMPORAL class.
6. **SFDB uses 1.21–2.71× more resident memory per fact than KG, and 1.97–2.01× more than KG-mem:** the latency advantages above are funded by keeping redundant per-open-set index structure in memory.

We do not claim SFDB is universally superior; the memory cost is real, inserts are not uniformly faster, the temporal classes show no design-level advantage once the storage layer is controlled for and are mildly *disadvantageous* on bounded TEMPORAL, and all of this is reported plainly rather than smoothed into a single directional headline.

11.15 Limitations of the Evaluation

The baseline and scale caveats that qualify every number above are collected in one place in Section 13 and analysed in Section 12.5 rather than repeated here; in short, the KG baselines are our own research prototypes rather than production RDF stores, rdflib is the only externally-authored system measured, and all measurements stop at 10^5 facts on a single node.

12 Discussion

12.1 What the Results Do and Do Not Show

The measurements in Section 11 show SFDB faster than the KG baseline on every *query* class measured, at every scale: 20–32× on LOOKUP, 54–99× on GLOBAL, 2.97–21× on CONTEXT, 1.23–1.66× on bounded TEMPORAL queries, and 1.78–2.17× on unbounded TEMPORAL queries. Insert throughput does not share that pattern: SFDB is slower than the KG baseline at the smallest scale measured and faster at every larger one (Section 11.2). The storage-layer control splits the query-class numbers into two different kinds of result: the LOOKUP, GLOBAL, and CONTEXT advantages survive against KG-mem (8.1–16×, 19–39×, and 1.59–9.0×) and are therefore properties of context-indexed atomic storage, while the temporal advantages do not survive at all — SFDB is at parity or measurably slower than KG-mem on bounded TEMPORAL at three of four scales, and mixed on unbounded TEMPORAL (0.40–0.89× and 0.59–1.08×, both ranges straddling parity) — and are therefore properties of answering queries without SQLite

round trips — something any engine can do, and the control does. CONTEXT is the query class the design is named for: both engines resolve it in time proportional to the result size rather than to N (neither does a full scan), so the distinction here is not asymptotic but architectural — SFDB reads the answer directly from a single open set the topology already pre-materialised at insert time (every fact registers into its own context’s open set and every ancestor’s), while KG-mem still has to enumerate the handful of distinct context buckets under the anchor and union their fact-id sets. Whether that constant-factor difference is large enough to matter empirically is exactly what Table 5 settles. This was not the paper’s original finding on the other three classes, on two separate occasions, and CONTEXT was not measured at all until this revision. An earlier iteration of this evaluation reported GLOBAL queries as SFDB’s weak point — up to three orders of magnitude slower than the KG baseline — because the query path enumerated topology open sets rather than a flat fact index. That earlier finding was correct as a measurement and wrong as an architectural conclusion: the open-set enumeration was an implementation choice, not a consequence of context-indexing, and a flat stalk index the engine already builds for LOOKUP queries answers GLOBAL queries directly once the query path is changed to use it (a change of roughly fifteen lines). A second, later iteration reported unbounded TEMPORAL queries as a genuine architecture-level limitation rather than a fixable implementation gap; that conclusion was also wrong, for the same underlying reason, and Section 12.2 describes the fix. We describe both earlier findings and their fixes here rather than silently updating the numbers, because the fact that the original measurements were real but the conclusions drawn from them were not is itself informative: an implementation-level gap in a code path can look, from the outside, exactly like a fundamental tradeoff until someone checks whether a cheaper path can be built alongside the existing one.

The tradeoff this paper does report, and that survives this scrutiny, is memory: SFDB uses $1.21\text{--}2.71\times$ more resident memory per fact than the KG baseline, and $1.97\text{--}2.01\times$ more than the dict-indexed control (Section 11.11). This is a structural consequence of the design, not an implementation oversight — every fact is registered in roughly eight to ten open sets plus four auxiliary indexes, and each registration holds a live reference into memory rather than a compact encoded row. Reducing it would require either coarsening the topology (fewer open sets per fact) or compressing the index structures (Section 14.7), both of which would likely give back some of the latency advantage.

We do not claim, and the data do not support, that the sheaf-inspired approach dominates a mature RDF store on every workload at every scale. Our KG baseline is a research prototype implemented by the same team as the sheaf engine and running against the same test harness, storage backend, and canonicalisation pipeline. It is a controlled reference, not a state-of-the-art. Any claim about how SFDB would fare against Jena TDB2 or Virtuoso must wait for a comparable evaluation, which we identify as future work.

12.2 Fixing the Unbounded Temporal Range

The bounded TEMPORAL results in Section 11 use a query bounded on both ends, which is the case the year-bucketed temporal index is built for; an open-ended query ("everything from year X onward") cannot be narrowed to a bounded set of buckets. An earlier version of this evaluation measured exactly that gap: at 10^4 facts, SFDB’s TEMPORAL latency degraded by roughly $40\times$ relative to the bounded case, to the point of being slower than the KG baseline, and we reported it as an architecture-level limitation — "not a fifteen-line fix, because there is no bounded candidate set to compute."

That claim does not survive scrutiny any better than the GLOBAL one did. There is no bounded candidate set within the *year-bucketed* index, but nothing requires the fix to live in that index: a second, flat, binary-searchable structure sorted by fact start/end timestamp, described mechanically in Section 8 and implemented in `sfdb.sheaf.indexes.TemporalIndex`, resolves an open-ended query in $O(\log M + n)$ rather than the $O(N)$ full-bucket fallback. This is the

exact fix Section 14.1 had proposed as future work before we implemented it; building it took an afternoon, not a research programme. With it in place, unbounded TEMPORAL queries are $1.78\text{--}2.17\times$ faster on SFDB than the KG baseline (Section 11.3) instead of slower — though the storage-layer control shows that this residual margin, like the bounded one, is mostly a storage-layer effect rather than a design win (Section 11.4). Both engines do comparable per-candidate interval filtering once the candidate set is identified. As with the GLOBAL case, the lesson is the same: the original measurement was real, the "architectural" framing around it was not.

12.3 When the Design Helps

The context-indexed design helps when the workload has any of the following properties. First, entity-anchored LOOKUP dominates the query mix: each such query is answered in $O(1)$ hash lookups rather than $O(k)$ triple joins, and the control shows this advantage belongs to the design, not the storage layer. Second, facts have arity above two or three: reification overhead grows linearly with arity in the KG baseline, while the SFDB engine stores each fact atomically. Third, queries scope naturally to sub-trees of the context poset: the CONTEXT benchmark class is a direct test of this, and SFDB is $2.97\text{--}21\times$ faster than KG and $1.59\text{--}9.0\times$ faster than the dict-indexed control at every scale measured (Section 11.3) — the topology's ancestor-open-set registration turns sub-tree scoping into a single pre-materialised lookup rather than a filter applied at query time. Fourth, the application requires verifiable data integrity across representations: the canonical layer and cross-engine verification harness make it cheap to run the same fact stream through both engines and catch inconsistencies. Fifth, and contrary to our own earlier expectation, unrestricted full-table scans are not a case where the reified design has an advantage: whole-fact retrieval via the flat stalk index beats per-fact reconstruction even against the dict-indexed control. Sixth, the LOOKUP and GLOBAL advantage is not an artefact of comparing against our own, possibly under-optimised baselines: it holds against three real, independently engineered production stores at every scale measured (Apache Jena TDB2, Section 11.6; OpenLink Virtuoso, Section 11.7; Ontotext GraphDB, Section 11.8), from a wide margin at small scale down to a still-clear $2.44\times$ (LOOKUP) and $8.8\times$ (GLOBAL) against Jena, $3.3\times$ (LOOKUP) and $15\times$ (GLOBAL) against Virtuoso, and $1.80\times$ (LOOKUP) and $6.3\times$ (GLOBAL) against GraphDB, at 100000 facts — and all three stores' reported latency includes an HTTP round trip SFDB's in-process call does not pay, which cuts against SFDB rather than inflating its advantage.

12.4 When the Reified Design Helps

The reified triple representation still wins in workloads that the SFDB design was not built for. Temporal-range scans are now, with the control in hand, the clearest example: once the storage layer is equalised, SFDB shows no consistent advantage on either TEMPORAL class and is measurably *slower* than the dict-indexed control on bounded TEMPORAL at three of four scales (Section 11.4), so a workload dominated by range scans over per-candidate filtering is better served by a reified design with a comparable index, not worse. The real Jena TDB2 comparison sharpens this further rather than contradicting it: Jena overtakes SFDB on TEMPORAL from 10^4 facts onward, at a ratio of $0.15\times$ (jena/SFDB latency, below 1 meaning Jena is faster by roughly seven times) at 100000 facts, despite paying the same HTTP overhead that works against it on the other three classes (Section 11.6). The Virtuoso and GraphDB comparisons each show the identical pattern independently: both overtake SFDB on TEMPORAL from the same scale onward, at ratios of $0.14\times$ (Section 11.7) and $0.16\times$ (Section 11.8) — none of the three margins identical, but the same crossover, the same query class, and the same scale threshold, from three independently engineered optimisers. So this is not only a storage-layer effect a simple dict-based control can recover, and it is not a property of one vendor's query planner: it is a case where mature production query optimisers genuinely outperform

SFDB’s Python-level candidate filtering once the candidate sets reach tens of thousands of rows, specifically on the one query class (TEMPORAL) where that outcome is not, per finding ten (Section 11.10), an artefact of SFDB’s own implementation leaving performance on the table. A workload dominated by TEMPORAL queries at that scale is a real production-engine-favoured case, not merely a KG-mem-favoured one. A small, insert-heavy workload is a second case: at the smallest scale measured, SFDB’s insert cost is $1.22\times$ the KG baseline’s, because the fixed cost of open-set registration has not yet amortised (Section 11.2). Workloads that are essentially binary—where facts genuinely are subject-predicate-object triples with no context, no arity, and no provenance—get no benefit from the sheaf structure and only pay the memory overhead of maintaining the extra indexes documented in Section 11.11, for no corresponding latency win. Any workload where memory is the binding constraint rather than latency favours the reified design’s more compact storage. Finally, integration with the extensive existing RDF and SPARQL tooling ecosystem is currently only possible on the KG side.

12.5 Threats to Validity

We identify four threats that a careful reader should weigh.

Implementation bias. Both in-house engines were written by the same authors, and both share code paths (serialisation, canonicalisation, planner infrastructure). We no longer have to speculate about how much this inflates the LOOKUP/GLOBAL/CONTEXT margin: Section 11.6, Section 11.7, and Section 11.8 compare against three real, independently engineered production stores (Jena TDB2, Virtuoso, and GraphDB), and SFDB remains faster on all three, on all three classes, at every scale, despite the comparison paying HTTP overhead that cuts against SFDB. The relative advantage on LOOKUP, GLOBAL, and CONTEXT against our own KG baseline is therefore an *upper bound* that all three comparisons confirm does not collapse entirely once a real engine is substituted — though the margins are smaller (as low as $2.44\times$ and $8.8\times$ against Jena, $3.3\times$ and $15\times$ against Virtuoso, and $1.80\times$ and $6.3\times$ against GraphDB, at 100000 facts, versus tens-to-hundreds-fold against our own baseline) and, on TEMPORAL, reverse outright once a real optimiser is doing the work instead of our KG-mem control’s dict-indexed reification. That the reversal happens at the same scale threshold and on the same query class against three independently engineered optimisers, from three different organisations, is evidence the pattern is a property of mature production query engines generally rather than one vendor’s specific implementation choices — three stores are still not an exhaustive survey of production RDF engines, but the pattern is now considerably better supported than after one or two. That this is a three-class win rather than a two-class one is itself a corrected reading, not the original one: finding ten (Section 11.10) found and fixed a genuine performance bug in SFDB’s own CONTEXT resolution that had been quietly costing it the CONTEXT margin throughout this evaluation, including against all three production stores.

Dataset bias. Every result up to this point uses synthetic datasets whose arity, context depth, and entity distribution we control. We no longer have to speculate about how much this matters: Section 11.9 repeats the full benchmark against 5,337 genuine Wikidata facts we did not construct, and Section 11.10 repeats it again against LUBM [9], the standard benchmark workload for RDF stores, at four scales up to 97,289 facts. Neither is a clean confirmation, though for different reasons. LOOKUP, GLOBAL, and CONTEXT reproduce the synthetic finding on both real datasets; TEMPORAL does not, and lands in a different place on each: it beats both baselines on Wikidata (the opposite of the synthetic-benchmark control result), and sits at roughly parity with both on LUBM’s two largest scales (neither the clean win of Wikidata nor the clear loss against KG-mem on synthetic data). Real knowledge graphs have long tails, correlated structure, and adversarial cases

that a controllable generator does not model by construction, and TEMPORAL landing in three different places across three datasets is direct evidence that one of the four margins this paper reports is genuinely sensitive to exactly that, in a way LOOKUP, GLOBAL, and CONTEXT are not. Building the LUBM comparison also surfaced finding ten (the bug inventory above): a genuine performance bug in SFDB’s CONTEXT resolution that LUBM’s topology shape (many large, mutually unrelated top-level contexts) exposed and neither the synthetic generator nor Wikidata’s data shape had. Two organic-or-standard datasets is still not a survey; DBpedia and BSBM/WatDiv-as-workloads remain untried.

Query bias. Our query set was designed with both engines in mind: one anchor-based class (LOOKUP), one no-anchor class (GLOBAL), one sub-tree class (CONTEXT), and one range-based class measured under two parameter settings (bounded and unbounded TEMPORAL). A different query mix would shift the aggregate numbers; Section 12.2 shows that varying a single parameter of the TEMPORAL class (whether the range is bounded) used to be enough to flip which engine is faster, before we added the index that closes that gap against the SQLite baseline. We do not claim the five classes measured here are exhaustive. Unlike an earlier revision of this paper, we also do not claim that the KG baseline’s scan-and-filter approach never wins: against the KG-mem storage-layer control, bounded TEMPORAL is exactly such a case, with SFDB slower at three of four scales (Section 11.4).

Scale. All measurements are at 10^5 facts or below. At the largest scale each fact expands to roughly 15 reified triples in the KG representation, so the KG engine is handling on the order of 1.5 million triples — past the entry point of standard RDF benchmarks, but far below the 10^8 -triple scales at which production stores are routinely evaluated. Our design choices are consistent with the complexity analysis in Section 8, but empirically our claims are bounded at 10^5 facts on a single node.

We also record, in the interest of showing our work rather than only its conclusion, that ten bugs, performance gaps, and measurement findings were surfaced during this evaluation, by eight different mechanisms, only one of which was the evaluation methodology catching itself automatically: the automated cross-engine harness (once); manual code inspection noticing an existing cheaper structure (three times); manual inspection triggered by building an unrelated external comparison that demanded exact semantics (once); a stress test failing outright at a larger scale than previously exercised (once); a targeted correctness test authored to validate an unrelated optimisation, which surfaced an independent bug as a side effect (once); repeating a seeded, supposedly deterministic benchmark in an independent process while building a second external comparison, which surfaced a measurement-methodology gap rather than a code bug (once); building a third external comparison, which surfaced a cross-engine SPARQL semantics divergence in the benchmark’s own shared query text rather than a code bug in any engine (once); and extending a standard benchmark workload with a topology shape (many large, mutually unrelated top-level contexts) none of the project’s own datasets had exercised, which surfaced a genuine performance bug that had been silently inflating a majority of this paper’s own CONTEXT measurements throughout the entire evaluation, invisible to the correctness-checking harness because it never affected results, only how much work produced them (once):

1. The sheaf engine’s LOOKUP path silently used neighbourhood (any-mention) match semantics instead of strict subject match. This one genuinely was caught by the cross-engine verification harness described in Section 7, after we tightened it from a positional, stringified comparison to an order-independent, type-preserving one: the two engines’ canonical result sets stopped agreeing, which is exactly what the harness is for.
2. The GLOBAL open-set enumeration inefficiency that produced an earlier draft’s headline negative result (Section 12.1) was *not* caught by the harness, because both engines already

agreed on which facts a GLOBAL query returned before the fix — they disagreed only on how long it took. It was found by inspecting the query path and noticing a cheaper index was already being maintained elsewhere for LOOKUP.

3. A linear scan standing in for a dictionary lookup in the KG engine’s identifier decoder was found the same way, by code inspection rather than by any automated check, and had been inflating the KG engine’s own reconstruction cost on every query class that touches reified triples.
4. The unbounded-TEMPORAL degradation (Section 12.2) was found and fixed the same way as the GLOBAL case: code inspection, not the harness, since it changed only how long a correct answer took to compute.
5. The benchmark’s “unbounded” TEMPORAL query was, for one revision, not actually unbounded: both engines share a bare-year convenience shorthand under which a start of “2023” with no end bound is silently re-bounded to the single year [2023, 2024), so the suite measured a re-bounded query while describing it as open-ended. The harness could not catch this — both engines implemented the same unintended semantics and agreed perfectly — and it was found by reading the shared query-parsing code while building the external rdflib comparison (Section 11.5), which needed the exact semantics spelled out in SPARQL. The fix issues a full ISO timestamp that the shorthand does not touch, and a regression test now pins the suite’s unbounded query to a genuinely open end.
6. The sheaf engine’s restriction-graph builder compared every pair of open sets to determine subset relations, an $O(|\text{open sets}|^2)$ construction that was invisible at the 10^2 – 10^4 -fact scales this evaluation used before this revision — tens of thousands of open sets at 10^5 facts made it run for over half an hour without completing. This was found by neither the harness nor code inspection but by scaling up: extending the evaluation to 10^5 facts to address the threat named in Section 12.5 (Scale) simply hung, which a fixed evaluation scope would never have surfaced. The fix replaces the pairwise sweep with a membership-driven construction (intersecting the topology’s point-to-open-set map) that is linear in total open-set memberships; a targeted test recomputes the old pairwise result as an oracle and asserts the two constructions agree.
7. Writing that test surfaced a second, unrelated bug in the same code path: the restriction graph is only ever grown by `add_edge` and was never cleared before a rebuild, so an edge recorded before an insert broke the subset relationship it depended on would survive in the graph forever. This does not affect any number reported in Section 11 — the benchmark inserts a fact stream once and only then issues queries against a fixed topology, so no run ever triggers a rebuild after the topology has changed since the previous one — but it is a genuine correctness gap in a code path SEMI-LOCAL query routing depends on for any workload that interleaves inserts and queries, which this evaluation’s own insert-once-then-query-many pattern cannot exercise. We fixed it (a `RestrictionGraph.clear()` called before every rebuild) and added a direct regression test rather than treat “the benchmark doesn’t hit it” as grounds to leave it.
8. Building the Apache Jena TDB2 comparison (Section 11.6) required repeating the 10^5 -fact CONTEXT benchmark in a separate process from the one that produced Section 11.3’s numbers, to run it alongside the Jena queries under identical conditions. The two measurements disagreed by $\sim 45\%$ (712ms versus 1044ms) despite both being ten-sample means of a seeded, deterministic benchmark with sub-2% within-run confidence intervals. This is not a code bug — both numbers are real measurements of the same code — but a gap in what the within-run statistics can see: they capture noise *within* one process’s ten samples, not variation *across* independent process launches. We reproduced the higher

figure with no other process running at all, ruling out contention with the concurrent Jena/Fuseki server as the explanation. A third independent process launch, needed to apply the query-portability fix in the next finding, produced a third value (989 ms) in the same span, reinforcing rather than undermining the finding.

9. Building the Virtuoso comparison (Section 11.7) surfaced a divergence not in SFDB’s code but in how the three external engines interpret the shared SPARQL query text itself. The TEMPORAL and unbounded TEMPORAL query text compares an RDF literal against another literal with `<` or `>`; against Virtuoso this silently returned every row unfiltered rather than a spec-consistent comparison, while the identical query text against Jena and rdflib correctly excluded the rows it should have and agreed with SFDB. We traced this to how Virtuoso resolves relational comparisons over untyped RDF literals and fixed it once, in the shared query text used by all three external comparisons, by wrapping the compared variable in `STR(...)` to force an unambiguous string comparison; all three external comparisons were re-verified to still agree with SFDB after the change. Unlike most of the findings in this list, this one is not a bug in code this project controls: it is a reminder that identical query text is necessary but not sufficient for a fair cross-engine comparison when the engines themselves disagree about SPARQL’s own semantics at the margins.
10. Building the LUBM standard-benchmark-workload comparison (Section 11.10) surfaced the most consequential finding in this list: a genuine performance bug in SFDB’s CONTEXT query resolution, present for the entire evaluation and invisible to every prior comparison. LUBM’s largest scale generates 265 mutually unrelated top-level contexts (one per university); a CONTEXT query anchored on one small, fixed department went from 0.68 ms to 350 ms as unrelated universities were added elsewhere, a 515× slowdown with the query’s own result size held constant at 68 facts the entire time. The cause (`src/sfdb/sheaf/optimizer.py`, `_classify_semilocal`): CONTEXT resolution found every fact whose exact context matched the query and unioned in every open set each belonged to, including the open set named for `context:world` — which every fact in the corpus belongs to whenever the query context is not the root — instead of taking the direct $O(1)$ lookup on the open set named `context:<c>`, which the architecture already materialises at insert time and which Section 8’s complexity analysis already assumed was what CONTEXT resolution did. Final results were always correct (a downstream filter narrows the over-broad candidate set back down before it reaches the caller), which is exactly why cross-engine verification never caught it across nine prior findings’ worth of scrutiny, and why the balanced synthetic generator never exposed it either: that generator grows a query’s own target subtree in lockstep with total corpus size, so the wasted work and the legitimate work scale together and look like the same line. Only a topology shape with many large, mutually unrelated top-level siblings — which none of this paper’s other datasets have — decouples them. The fix targets `context:<c>` directly; `tests/sheaf/test_context_query_scope.py` pins it, confirmed to fail against the pre-fix code. Because CONTEXT queries appear in every comparison this paper reports, this finding required re-running and re-verifying every benchmark in Section 11 rather than a local patch; two earlier claims in this paper (the eighth finding above, and a CONTEXT-loses-to-KG-mem reading in Section 11.9) are retracted as a direct consequence, and the corrected numbers show SFDB winning CONTEXT robustly everywhere it is measured rather than losing it from 10^4 facts onward against three production stores. We report the retractions alongside the fix rather than quietly substitute corrected numbers into the earlier sections and leave the narrative as though CONTEXT had always been this fast. The fix was subsequently re-verified under an even harder test than any single comparison in this paper already ran: LUBM data against all three external stores at once

(Section 11.10.1), varying the dataset and the comparison engine simultaneously. The corrected CONTEXT advantage held there too, at every scale, against every store.

All ten findings are reflected in the numbers reported in Section 11, Section 11.6, Section 11.7, Section 11.8, and Section 11.10. That only the first was actually caught by an automated check is itself a threat to validity worth naming plainly: the verification harness is real and it does its job, but it checks result equivalence, not performance or intent, and it cannot catch an inefficiency both engines agree on, a specification bug both engines implement identically, or a correctness gap in a code path the benchmark’s own access pattern never exercises — the tenth finding is the sharpest instance of exactly that last case: a bug that silently inflated a majority of the paper’s own CONTEXT numbers throughout the whole evaluation, past nine prior rounds of scrutiny, because it never touched a single returned result. The sixth finding adds a further point: a fixed evaluation scope can hide a scalability cliff that only appears once you actually push past it, which is itself an argument for the 10^5 -fact scale this revision adds over the 10^4 ceiling of the one before it. The eighth, as originally stated, claimed something different: that even a seeded, deterministic benchmark is not automatically reproducible *across* process launches to the precision its own within-run confidence intervals suggest. We no longer stand behind that specific claim — Section 11.6 retracts it, because the numbers it rested on turned out to be measurements of the bug the tenth finding fixes, not of genuine process-level variance — and we are explicit about that rather than let a wrong methodological lesson stand uncorrected alongside the finding that undermines it. The ninth adds a point that does still stand: "identical query text across engines" is a methodological commitment worth stating plainly (Section 11.6), but it is not by itself a guarantee of a fair comparison, since the engines being compared can silently disagree about what that text means. The tenth adds the sharpest point of all: a bug that never affects a single returned result can still be a bug worth an entire evaluation’s recomputation, and "the harness verifies correctness" is not the same claim as "the harness verifies the paper’s numbers are what they should be."

12.6 Negative Results

Four negative results deserve to be stated plainly, because they discipline the paper’s overall claim. (An earlier version of this section reported a different one in place of the second below — TEMPORAL degradation under an open-ended range — which Section 12.2 describes fixing; we note that history rather than silently rewrite it.)

The first is the memory cost quantified in Section 11.11: SFDB uses $1.21\text{--}2.71\times$ more resident memory per fact than the KG baseline and $1.97\text{--}2.01\times$ more than the dict-indexed control. This is the tradeoff that funds every latency advantage reported in this paper, and it is intrinsic to the design rather than an implementation artefact: redundant per-open-set registration is what makes the stalk and temporal indexes cheap to query.

The second is new in this revision and comes directly from the storage-layer control: the TEMPORAL advantages over the SQLite baseline do not survive against KG-mem, and on bounded TEMPORAL the direction reverses at most scales — SFDB is slower than the dict-indexed control at three of the four scales measured ($0.40\text{--}0.89\times$ bounded, $0.59\text{--}1.08\times$ unbounded, both ranges straddling parity). Context-indexed storage buys nothing measurable for range scans whose cost is dominated by per-candidate filtering, and on the bounded case costs a little; the earlier draft’s temporal “wins” were storage-layer effects. We consider surfacing this worth more than the wins it retracts: it is precisely the kind of finding the control exists to produce.

The third, also new in this revision, is that insert throughput does not favour SFDB unconditionally: at the smallest scale measured (100 facts) SFDB is $1.22\times$ *slower* than the write-through KG baseline, and only becomes faster from 1 000 facts onward (Section 11.2). The fixed per-insert cost of registering a fact into eight to ten open sets does not amortise until the fact stream

is large enough, which means the design’s insert advantage is a statement about scale, not an unconditional property of the engine.

The fourth is the vacuousness of the sheaf condition in our setting (Section 6.4): the system is, strictly, a presheaf database with a defensive consistency check, not a sheaf database in any technical sense that would constitute a contribution on its own. This does not diminish the practical value of the design: what makes the design work is the context poset, the stalk index, the canonical intermediate representation, and the disciplined restriction vocabulary that comes with them. It does mean that any framing of the paper as a sheaf-theoretic breakthrough would be dishonest, and we do not adopt that framing.

13 Limitations

We describe the design’s limitations honestly and in one place so that a reader can weigh what the paper does and does not claim.

13.1 Scale and Deployment

The current prototype is single-node and in-memory. All measurements are at scales of 100 000 facts or below; behaviour at larger scales is extrapolated from the complexity analysis of Section 8 and not directly measured. Global-section computation in the worst case is $O(|\mathcal{C}| \cdot N^2)$, which imposes a natural ceiling on the size of context poset that the system can support on a single node. This ceiling is compounded by a concretely measured one: Section 11.11 reports $1.21\text{--}2.71\times$ higher resident memory per fact than the KG baseline, so the single-node memory budget is consumed faster by SFDB than by an equivalent reified store, independent of the gluing worst case. Distributing the store across nodes would require a partitioning strategy for the context poset and a distributed restriction-map protocol; both are outside the scope of this paper.

13.2 Mathematical Assumptions

The formal model assumes a finite context poset equipped with the Alexandrov topology. This is the right setting for the intended workload, but it is also the setting in which the sheaf condition holds vacuously. Nothing in the paper claims a non-trivial sheaf theorem. The gluing operation in the implementation is a defensive consistency check rather than a step in a proof; in the presence of genuinely contradictory local data (which our schema currently forbids), the algorithm would report the conflict rather than resolve it.

A further structural assumption deserves to be stated explicitly: the dot-separated path encoding makes the context hierarchy a *tree* (each context has exactly one parent), so incomparable contexts never share a descendant and meets of incomparable contexts do not exist (Section 6.3). Real-world context structures are not always tree-shaped — a policy that holds in the intersection of two jurisdictions, or an experimental condition scoped by both a protocol and a lab, is naturally a DAG node with two parents. Supporting DAG-shaped posets would preserve the Alexandrov construction (which needs only a preorder) but would invalidate the prefix-string encoding, the longest-common-prefix join, and the single-ancestor-chain $O(d)$ insert path, so we scope it out here and note it as a design extension rather than pretending the tree assumption is costless.

13.3 Implementation Limitations

The prototype does not currently support concurrent transactions, on-disk persistence beyond the built-in SQLite backing store, distributed execution, streaming or incremental fact ingestion, or SPARQL compliance. An earlier draft of this paper described the query language as "a small subset of SPARQL"; that overstated it. It is a bespoke query DSL whose surface syntax

borrowing SPARQL’s **SELECT**/**WHERE** keywords and **?var** notation but does not parse SPARQL’s graph-pattern braces or its **FROM** clause, together with a few sheaf-native operations (restriction, stalk lookup). The typed **Query** object used internally by both engines and by the benchmark harness (Section 11) does not depend on this text surface. A real SPARQL front end and a proper transactional layer are engineering work that would be needed for any real deployment but that would not change the paper’s structural findings.

13.4 Baseline and Benchmark Limitations

The KG baseline is a research prototype implemented by the same team as the sheaf engine. It is a controlled reference point, not a state-of-the-art RDF store. Both engines share the same serialisation and canonicalisation layers, and both persist inserts to in-memory SQLite — but the *query* paths differ in more than indexing strategy: the KG baseline answers queries through SQLite, while SFDB answers them from in-process Python indexes and touches SQLite only on the write path. A comparison of the two alone therefore conflates two variables — context-indexed storage versus reification, and dict indexes versus SQLite round trips. The evaluation separates them with the KG-mem control (Section 11.4): a variant of the KG baseline with identical reification and query logic whose dictionary tables and triple indexes are Python dicts, so that the SFDB-vs-KG-mem ratios isolate what context-indexed storage buys with the storage-layer difference removed.

Both the KG baseline and the KG-mem control are, however, still research prototypes by this project’s authors, and that ceiling has since been addressed directly rather than left as a caveat: three real production stores, Apache Jena TDB2, OpenLink Virtuoso, and Ontotext GraphDB, are each compared on all five query classes at every scale (Section 11.6, Section 11.7, Section 11.8), which is the natural next step this section previously called out as future work. The result is a genuine crossover, reproduced independently against all three — SFDB faster on LOOKUP/GLOBAL/CONTEXT, the production store faster on TEMPORAL from 10^4 facts — not a clean win either way, which we take as more credible than a one-sided result would have been, and the agreement between three unrelated optimisers as evidence the pattern generalises well beyond one vendor. What remains open is breadth, not existence: three stores is not an exhaustive survey, and LUBM [9] (Section 11.10) has since been run both against the three in-house engines and against all three external stores at once (Section 11.10.1), reproducing the identical pattern against every one; BSBM [5] and WatDiv [2] have not been tried at all, nor has LUBM at the much larger scales published LUBM results use. The dataset-bias half of this concern has since been addressed directly rather than left as a caveat, and the answer is not the tidy one the LOOKUP/GLOBAL results alone would suggest: a real-world case study (Section 11.9, 5,337 genuine Wikidata facts) and the LUBM comparison both reproduce the LOOKUP, GLOBAL, and CONTEXT advantage, while TEMPORAL lands somewhere different on each — beating both baselines on Wikidata, roughly at parity with them on LUBM’s two largest scales, neither matching the synthetic benchmark’s own result. The synthetic datasets, while parameterised, do not exhibit all pathologies of organic knowledge graph data, and these two comparisons are direct evidence of exactly that for TEMPORAL specifically — and, for CONTEXT, direct evidence of something else: LUBM’s topology exposed a genuine performance bug in SFDB’s own query resolution (Section 12.5’s tenth finding) that neither the synthetic generator nor Wikidata’s data shape had triggered, which is itself an argument for dataset diversity in an evaluation independent of what any single comparison’s headline result turns out to be.

A further limitation of the KG-mem control deserves to be named rather than left implicit: KG-mem removes the SQLite round trip, but it still *reconstructs* each fact from its decomposed triples on every query (`_reconstruct_fact` in the KG engine, unchanged by the storage-layer swap) rather than returning a stored whole-fact object the way SFDB’s stalk index does. Nothing in principle stops a reified store from also maintaining a fact-id-to-materialised-fact cache

alongside its triple indexes — which is architecturally close to what SFDB’s stalk index is — at which point some of the LOOKUP, GLOBAL, and CONTEXT advantage that survives the KG-mem control in Section 11.4 would likely narrow further, for the same memory cost SFDB already pays. This concern is somewhat allayed by the Jena, Virtuoso, and GraphDB results: all three are exactly such mature, materialising, professionally engineered indexes, independently built by three different organisations, and all three still lose to SFDB on LOOKUP, GLOBAL, and CONTEXT at every scale measured despite also paying HTTP overhead SFDB does not — so the advantage on those three classes does not appear to be merely an artefact of comparing against an under-optimised control, or of one vendor’s specific engineering choices. The control isolates the SQLite-versus-dict variable cleanly; it does not isolate reconstruction-from-parts versus retrieval-of-a-whole, which is a second, unremoved variable the LOOKUP, GLOBAL, and CONTEXT numbers still partially reflect. (CONTEXT’s advantage is dominated by candidate-set lookup rather than reconstruction, so this caveat bears on it more weakly than on LOOKUP and GLOBAL, but every result row is still reconstructed once found.) We do not add a second control for this in the current evaluation and name it here as an open question rather than letting the existing control’s precision imply more than it isolates.

14 Future Work

Several concrete extensions would improve either the design or the credibility of the evaluation. We list them in the order we consider useful; the first is no longer future work, and we keep the section to record what it was and what came of it.

14.1 A Flat Temporal Index for Unbounded Ranges — Implemented

An earlier revision of this section proposed a flat, sorted temporal index alongside the year-bucketed one, so that an open-ended range query could binary-search to its starting point instead of degrading to a full-bucket fallback. That proposal has since been implemented (`sfdb.sheaf.indexes.TemporalIndex.facts_ending_after` and its symmetric counterpart `facts_starting_b`) and benchmarked; Section 12.2 tells the full story. We keep the entry, marked implemented, because a future-work item that turned out to be an afternoon’s change rather than a research programme is itself evidence about how easily this class of gap is mistaken for a design ceiling.

14.2 Real-World Datasets and Standard Benchmarks — Done

An earlier revision of this section named the evaluation’s reliance on synthetic data as a gap. Section 11.9 has since run the full benchmark against a genuine, organic dataset (Wikidata office-holder facts we did not construct), and Section 11.10 against LUBM [9], the standard benchmark workload for RDF stores, scaled up to 97,289 facts. Both moved the evaluation from speculation to evidence, in different ways. LOOKUP, GLOBAL, and CONTEXT hold on both; TEMPORAL, the one query class this paper’s synthetic benchmark already reported as sensitive to the storage-layer control, turns out to be genuinely sensitive to the dataset itself too — it beats both in-house baselines on Wikidata and sits at roughly parity with them on LUBM’s two largest scales, neither matching the synthetic result nor each other. LUBM’s topology (many large, mutually unrelated top-level contexts, unlike either the synthetic generator’s balanced tree or Wikidata’s one-populous-country-among-many shape) also surfaced a genuine performance bug in SFDB’s own CONTEXT resolution (Section 12.5’s tenth finding), fixed and re-verified across the entire evaluation. What remains is breadth on the dataset side (DBpedia, a different corner of Wikidata) and, on the standard-benchmark side, running LUBM against an external store rather than only this project’s own three in-house engines, and at the much larger scales published LUBM results use — see Section 14.3.

14.3 Integration With Production RDF Stores — Done

An earlier revision of this section named a comparison against a real, disk-backed production RDF store as “the single highest-leverage item on this list,” since every simulated reviewer persona this project consulted (`research/reviewer_simulation.md`) raised it first. That comparison has since been run three times, against three independently engineered stores: Section 11.6 reports all five query classes against a real Apache Jena TDB2 store (accessed over HTTP via a Fuseki server, exactly as an external client would), Section 11.7 reports the identical methodology against a real OpenLink Virtuoso store, and Section 11.8 reports it again against a real Ontotext GraphDB store, at every scale this paper measures, with identical result counts on every row against all three. The finding is a genuine crossover rather than a clean win, and it reproduces independently across all three stores: SFDB remains faster than all three on LOOKUP, GLOBAL, and CONTEXT at every scale, while all three overtake SFDB on TEMPORAL from 10^4 facts onward. We regard this as a more credible and more informative result than a one-sided win would have been, and the agreement between three unrelated optimisers, from three different organisations, as strong evidence the pattern is a property of mature production query engines generally rather than one vendor’s implementation — which replaces the earlier framing of this item as unstarted future work, the question the second revision of this section posed of whether a two-store agreement would hold for a third, and the original question of whether a single comparison would generalise at all.

What remains open: three stores is not an exhaustive survey of production RDF engines, so a fourth or fifth comparison (e.g. Blazegraph or Stardog, or a labeled property graph store such as Neo4j, which would additionally require a schema translation rather than only a new adapter — Section 2 discusses why) would strengthen the generalisation claim further, though with diminishing marginal value at this point. Standard benchmark workloads have since also been run against all three external stores, not only against this project’s own in-house engines: LUBM [9] (Section 11.10.1) reproduces the identical LOOKUP/GLOBAL/CONTEXT-robust, TEMPORAL-crossover pattern against Jena, Virtuoso, and GraphDB, at scales up to 97289 facts, with both the dataset and the comparison engine varied at once — the one axis-combination no other comparison in this paper tests. What remains is scale (LUBM has not been run at the much larger scales published LUBM results use) and breadth: BSBM [5] and WatDiv [2] have not been tried at all. That remains future work, but with the core “have we ever measured against a real store” question answered, three times over, and “have we ever measured against a standard workload, against a real store, varying both at once” question also answered, three times over.

14.4 An Incidence-Algebra Query Layer

Our survey of neighbouring formalisms identified incidence algebras over locally finite posets [18] as the closest mathematical neighbour to the context-poset design (Section 2): the context poset is exactly the kind of structure on which Möbius-inversion-style aggregation is defined, which suggests a principled query layer for roll-up and drill-down across context hierarchies. We deliberately did not pursue this here — it is a follow-up contribution, not a missing piece of the present evaluation — but it is the extension we consider most likely to add new query capability rather than only new speed.

14.5 Incremental Restriction and Global-Section Maintenance

Restriction maps are computed on demand and cached. Incremental maintenance—updating cached restriction maps and the global-section cache on insert rather than invalidating them—would improve query latency in workloads that mix inserts and queries.

14.6 Concurrent and Distributed Execution

Global-section computation is embarrassingly parallel across independent context sub-trees. A parallel implementation would remove the single-node scalability ceiling. A distributed implementation would additionally require a partitioning strategy for the context poset and an inter-node restriction-map protocol.

14.7 Compression of Contextually Redundant Data

Section 11.11 reports that SFDB uses $1.21\text{--}2.71\times$ more resident memory per fact than the KG baseline, because each fact is registered in roughly eight to ten open sets plus four auxiliary indexes. Sections that share a common context or that agree under restriction on a large subtree admit context-aware compression schemes that are not available to a flat triple store: rather than storing a full section reference per open set, a fact could store one canonical reference plus a small bitset of the open sets it belongs to, reconstructed on read. Realising this would directly reduce the memory overhead identified in Section 11.11, which is currently the paper’s primary reported cost.

14.8 Hybrid Storage — Revisited

An earlier version of this section proposed a hybrid system that would store facts under both representations and let the optimiser choose the access path per query, on the premise that this would combine SFDB’s LOOKUP advantage with a GLOBAL advantage that belonged to the KG baseline. That specific premise no longer holds: Section 11 reports SFDB faster than the KG baseline on GLOBAL as well as LOOKUP. The storage-layer control did surface a genuine case where the reified design wins: on bounded TEMPORAL, SFDB is measurably slower than the dict-indexed KG-mem control at every scale measured ($0.40\text{--}0.89\times$, Section 11.4), which is precisely the kind of gap that would motivate a second, reified access path for range-scan-heavy workloads. We still do not prioritise building it here, for a narrower reason than the earlier premise offered: the margin against KG-mem on bounded TEMPORAL narrows as scale grows rather than widening, from roughly two-and-a-half-fold at the smallest scale measured down to close to parity at the largest ($0.40\text{--}0.89\times$, a ratio below 1 meaning SFDB is slower); the widest gap sits at sub-millisecond absolute latencies for both engines, the same measurement-floor regime Section 11.2 already treats cautiously for insert timing. SFDB also remains faster than the *raw* KG baseline on every TEMPORAL row measured (Section 11.3), so the case for a second representation would rest on beating a control that itself does not persist to disk, against a margin that is small where it is trustworthy. Doubling the memory cost identified in Section 11.11 — the paper’s central tradeoff — to close a gap of that size is not an obviously good trade. A hybrid design remains worth revisiting if a real workload shows the gap widening at scale, which this evaluation’s four data points do not yet show one way or the other.

15 Conclusion

We described the Semantic Fact Database, a prototype context-indexed storage system for semantic facts. The system stores each fact atomically as a section over a finite context poset and answers entity-anchored queries through a stalk index that returns complete facts in a single hash lookup. A companion reified triple-store baseline is provided over the same canonical schema, and a constructive semantic-equivalence proof binds the two engines together, checked at run-time by an automated harness: every insert and every query is checked, online, for canonical result equality.

The evaluation, on five query classes and insert throughput at 4 scales up to 10^5 facts, yields a decomposed result and a clear tradeoff. Against the write-through KG baseline, SFDB

is faster on every query class measured at every scale: entity-anchored LOOKUP by 20–32 $\times$, unrestricted GLOBAL queries by 54–99 $\times$, CONTEXT queries — the class the design is named for — by 2.97–21 $\times$, and the two temporal-interval classes by 1.23–1.66 $\times$ and 1.78–2.17 $\times$. Insert throughput does not share that pattern: SFDB is slower than KG at the smallest scale measured and faster at every larger one. The storage-layer control splits the query-class headline in two: the LOOKUP, GLOBAL, and CONTEXT advantages survive against the dict-indexed KG-mem control (8.1–16 $\times$, 19–39 $\times$, and 1.59–9.0 $\times$) and are properties of storing facts atomically under a context index, while the temporal margins do not survive at all — SFDB is at parity or measurably slower than KG-mem on bounded TEMPORAL at three of four scales — and are storage-layer effects, which we report as a negative result rather than reframe. Four external reference points anchor the comparison outside our own code: rdflib, on every query class with result counts verified, and — the single highest-leverage gap this project’s own reviewer simulation and roadmap identified — three real, disk-backed production stores, Apache Jena TDB2, OpenLink Virtuoso, and Ontotext GraphDB, independently engineered by three different organisations and accessed over SPARQL/HTTP exactly as an external client would (Section 11.6, Section 11.7, Section 11.8). Against all three, SFDB remains faster on LOOKUP, GLOBAL, and CONTEXT at every scale, and is overtaken on TEMPORAL from 10^4 facts onward: a genuine, scale-dependent crossover, not a clean win or a hedge, reproduced independently against three unrelated optimisers, and the first time this project has measured itself against production engines rather than only its own research prototypes. Two further comparisons push this further still, and not only in directions a tidier narrative would have chosen. A real-world case study (Section 11.9) against 5,337 genuine Wikidata facts and the LUBM standard benchmark workload (Section 11.10) up to 97,289 facts both reproduce the LOOKUP/GLOBAL/CONTEXT finding cleanly, while TEMPORAL lands somewhere different each time — beating both in-house baselines on Wikidata, roughly at parity with them on LUBM’s largest scales — evidence that TEMPORAL alone among the four query classes is genuinely sensitive to a dataset’s actual shape. Building the LUBM comparison surfaced something larger than a data-shape finding: a genuine performance bug in SFDB’s own CONTEXT query resolution, present for the entire evaluation and exposed only by a topology shape (many large, mutually unrelated top-level contexts) none of this paper’s other datasets have, which required re-running and re-verifying every CONTEXT number this paper reports and retracting two earlier claims built on the unfixed numbers. We report the retraction alongside the fix rather than quietly substitute corrected numbers and leave the earlier narrative standing. The tradeoff that funds the surviving advantages is memory: SFDB uses 1.21–2.71 $\times$ more resident memory per fact than the reified-triple baseline, and 1.97–2.01 $\times$ more than the control. We report this plainly. We also report — once, in full, in Section 12.5 — that ten bugs, performance gaps, and measurement findings were surfaced while preparing this evaluation, by eight different mechanisms, only one of which was the automated verification harness catching itself. We treat the fact that our own evaluation needed ten rounds of correction — one of them large enough to change the paper’s own central crossover claim from two query classes to one — as part of the paper’s evidence for the value of that scrutiny rather than as an embarrassment to suppress.

The formal apparatus in the paper is that of presheaves on a finite poset. We do not claim a novel sheaf-theoretic theorem: on the Alexandrov topology that fits our setting, the sheaf condition holds vacuously and could not serve as a technical contribution. The value of the framework is operational. It gives us a disciplined vocabulary for restriction, stalks, and global sections that maps directly onto the components of the implementation, and it gives us a canonical intermediate representation that makes cross-engine equivalence a checkable property rather than an aspiration.

We do not claim that the design supplants production RDF stores, and the Jena TDB2, Virtuoso, and GraphDB comparisons are direct evidence for why not: SFDB loses to all three on one of the five query classes measured once the working set is large enough. Our KG base-

line remains a controlled research implementation, and the LOOKUP/GLOBAL/CONTEXT speedups against it in particular are best interpreted as a measurement of what disciplined context-indexed storage buys over a strict reification-plus-joins design at comparable engineering effort, now corroborated by three independent production-store results and the LUBM standard workload rather than merely asserted. Extending the evaluation to further standard benchmark workloads (LUBM at the much larger scales published LUBM results use, BSBM, WatDiv) remains future work, to check whether the crossover generalises further still. The artifact and reproducibility manifest for every result in the paper are released with the paper.

16 Artifact Availability and Reproduction

The complete source code, datasets, benchmark harness, and reproducibility manifests are released under the MIT License at <https://github.com/BlackSamuron0305/semantic-fact-db>. Table 16 records the exact commit, interpreter, lock-file hash, dataset checksum, and seed behind every number in this paper; `results/paper_suite_reproducibility.json` carries the same manifest alongside the raw benchmark output.

Table 16: Reproducibility metadata for the reported run.

Property	Value
Git commit	b0492ee
Python version	3.14.4
uv lock hash	f5166dc09722c41f
Dataset checksum	d19da6076a71595e
Random seed	42
Generation date	2026-08-17T07:52:59.957689+00:00

From a clean clone, the following commands reproduce every number and figure in this paper except the Jena TDB2, Virtuoso, and GraphDB comparisons and the three LUBM-vs-external-store runs, all four of which require a separately running external service (see below) (`sfdb benchmark` with no `-size` argument runs all four scales — 10 timed runs and 2 warm-up iterations per query class per Section 11.1 — and exits non-zero if any cross-engine verification fails):

```
uv sync --group dev
uv run pytest                # full test suite should pass
uv run sfdb benchmark        # populates results/ (3 engines)
uv run python scripts/rdfli
b_reference.py               # external rdfli
b_reference point
uv run python scripts/wikidata_case_study_benchmark.py # case study
uv run python scripts/lubm_benchmark.py                # LUBM vs. the 3 in-house engines
uv run python scripts/generate_tables.py                # tables + reproducibility macros
uv run python scripts/generate_figures.py               # figures
cd paper && latexmk -pdf main.tex                       # main.pdf
```

The case-study benchmark reads a cached Wikidata snapshot shipped in the artifact rather than querying Wikidata live. A separate fetch script (under `scripts/`) refreshes that snapshot but is an explicit, optional step, not part of the reproduction sequence above.

The Apache Jena TDB2 comparison (Section 11.6) requires a separately running Fuseki server, since it is a genuine external process rather than code this artifact ships and controls. Requires a Java 17+ runtime, which the reproduction script does not install:

```
# One-time: download and start a Fuseki server with a fresh TDB2 dataset
```

```
curl -LO https://dlcdn.apache.org/jena/binaries/apache-jena-fuseki-6.2.0.zip
unzip apache-jena-fuseki-6.2.0.zip && cd apache-jena-fuseki-6.2.0
java -jar fuseki-server.jar --loc=./tdb2data --update --port 3030 /sfdb &
```

Then, from the repository root:

```
uv run python scripts/jena_reference.py      # populates results/
uv run python scripts/lubm_external_reference.py --store jena # LUBM vs. Jena
uv run python scripts/generate_tables.py     # regenerates jena tables too
```

The OpenLink Virtuoso comparison (Section 11.7) likewise requires a separately running Virtuoso instance. The simplest route is the official Docker image:

```
# One-time: start a Virtuoso open-source-edition container
docker run -d --name virtuoso-sfdb -p 8890:8890 -p 1111:1111 \
  -e DBA_PASSWORD=sfdb_bench_2026 \
  openlink/virtuoso-opensource-7:latest
```

Then, from the repository root:

```
uv run python scripts/virtuoso_reference.py  # populates results/
uv run python scripts/lubm_external_reference.py --store virtuoso # LUBM vs. Virtuoso
uv run python scripts/generate_tables.py     # regenerates virtuoso tables too
```

The Ontotext GraphDB comparison (Section 11.8) likewise requires a separately running GraphDB instance. The simplest route is the official Docker image; unlike Virtuoso, no authentication or manual repository setup is needed:

```
# One-time: start a GraphDB free-edition container
docker run -d --name graphdb-sfdb -p 7200:7200 ontotext/graphdb:10.7.3
```

Then, from the repository root (auto-creates its repository):

```
uv run python scripts/graphdb_reference.py  # populates results/
uv run python scripts/lubm_external_reference.py --store graphdb # LUBM vs. GraphDB
uv run python scripts/generate_tables.py     # regenerates graphdb tables too
```

To reproduce independent of the host’s Python version or operating system, a Dockerfile at the repository root builds and runs the test suite in a container:

```
docker build -t sfdb .
docker run --rm sfdb # runs the test suite
docker run --rm sfdb uv run sfdb benchmark # runs the benchmark suite
```

The 14 tests that require a live Jena, Virtuoso, or GraphDB instance skip inside the isolated container rather than fail. The image does not bundle TeX Live or the external stores themselves.

Source code, data generators, and benchmark harnesses are released under the MIT License.

A Supplementary Formal Material

This appendix collects formal content referenced from the main text but not required to follow it: definitions that are not cited by number elsewhere in the paper (Appendix A.1), and the standard theorems that follow from the formal model of Section 6 but that this paper does not build on directly (Appendix A.2). The one theorem the paper’s argument actually depends on, cross-engine equivalence (Theorem 7.1), is proved in place in Section 7 rather than here.

A.1 Additional Definitions

Definition A.1 (Entity). An *entity* is a distinguished identifier $\varepsilon \in \mathcal{E} \subseteq \mathcal{I}$ with:

- A unique identifier $\text{id}(\varepsilon) \in \mathcal{I}$.
- A semantic type $\text{type}(\varepsilon) \in \mathbf{SemanticType}$.
- An optional name $\text{name}(\varepsilon)$.
- Optional attributes $\text{attrs}(\varepsilon) \subseteq \mathcal{V}^*$.

If $f = (i, s, r, \vec{o}, c, k, m)$ is a fact, then $s \in \mathcal{E}$.

Definition A.2 (Relation). A *relation* is a distinguished identifier $\rho \in \mathcal{R} \subseteq \mathcal{I}$ with:

- A unique identifier $\text{id}(\rho) \in \mathcal{I}$.
- An arity $\text{arity}(\rho) \geq 0$.
- Slot types $\text{slots}(\rho) \in \mathbf{SemanticType}^*$.

A fact $f = (i, s, r, \vec{o}, c, k, m)$ is well-typed iff $\text{arity}(r) = |\vec{o}|$ and each ω_j conforms to $\text{slots}(r)_j$.

Definition A.3 (Open Set). An *open set* $U \in \mathcal{T}$ is a subset of X that is declared open. In the Alexandrov topology induced by $(\mathcal{C}, \leq)$, every down-set $\downarrow c = \{d \in \mathcal{C} \mid d \leq c\}$ is open. To each context c we associate $U_c = \{f \in \mathcal{I} \mid \text{context}(f) \leq c\}$. Properties: $U_\top = X$; the assignment is monotone ($c_1 \leq c_2 \implies U_{c_1} \subseteq U_{c_2}$); intersections follow Theorem A.6 below ($U_{c_1 \wedge c_2}$ when the meet exists, $\emptyset$ for incomparable contexts).

Definition A.4 (Knowledge State). A *knowledge state* is $\Sigma = (X, F)$ where X is a finite topological space of fact identifiers and F is a presheaf on X assigning facts to open sets. State transitions: insert $\Sigma \mapsto \Sigma \cup \{f\}$, delete $\Sigma \mapsto \Sigma \setminus \{f\}$, update $\Sigma \mapsto (\Sigma \setminus \{f\}) \cup \{f'\}$.

Definition A.5 (Canonical Query). A *canonical query* is $Q = (\tau, s, r, \vec{o}, c, k_{\min}, n_{\max})$ where: $\tau \in \{\text{FACT}, \text{WALK}, \text{JOIN}, \text{GLUE}\}$ is the query mode, $s \in \mathcal{E} \cup \{*\}$ is the subject (wildcard $*$ matches all), $r \in \mathcal{R} \cup \{*\}$ is the relation, $\vec{o} \in \mathcal{V}^* \cup \{*\}$ is the object pattern, $c \in \mathcal{C} \cup \{*\}$ is the context, $k_{\min} \in [0, 1]$ is the minimum confidence, $n_{\max} \in \mathbb{N}$ is the result limit. The denotation $\llbracket Q \rrbracket_\Sigma$ is the set of facts matching Q .

A *canonical result* is $R = \llbracket Q \rrbracket_\Sigma$, a set of canonical facts. Section 7 defines *semantic equivalence* on canonical results (Definition 7.2) as literal equality of order-independent, type-preserving canonical result sets, which is what the benchmark harness checks and what Theorem 7.1 proves; that is the only definition of result equivalence used anywhere in this paper.

A.2 Theorems and Proofs

The definitions above and in Section 6.3 satisfy the standard consequences one would expect of a presheaf formalisation. None of the theorems below is cited by number elsewhere in the paper; we record them here as a completeness check on the formal model rather than as load-bearing results, in contrast to Theorem 7.1 (Section 7), which the paper’s central claim actually depends on.

A.2.1 Semantic Preservation

Theorem A.1 (Semantic Preservation / Round-trip Correctness). Let $T : \mathcal{F} \rightarrow \mathcal{T}(\mathcal{F})$ be the triple decomposition function (reifying an n -ary fact into $2 + n$ triples), and $R : \mathcal{T}(\mathcal{F}) \rightarrow \mathcal{F}$ be the reconstruction function. For any semantic fact $f \in \mathcal{F}$:

$$R(T(f)) = f.$$

Proof Sketch. Decomposition T creates triples $(s, \text{rdf} : \text{type}, \text{rdf} : \text{Statement})$, $(s, \text{rdf} : \text{subject}, f.\text{subject})$, $(s, \text{rdf} : \text{predicate}, f.\text{relation})$, and $(s, \text{rdf} : \text{object}_j, f.\text{objects}_j)$ for each j . Reconstruction R groups triples by statement ID s and inverts each step. Since T is injective and R is its left inverse on $\text{im}(T)$, the round-trip is identity. $\square$

A.2.2 Functoriality of Restriction

Theorem A.2 (Restriction Functoriality). The restriction maps $\rho_{d,c} : F(d) \rightarrow F(c)$ for $c \leq d$ satisfy:

$$\rho_{c,c} = \text{id}_{F(c)}, \quad \rho_{e,c} = \rho_{d,c} \circ \rho_{e,d} \quad (c \leq d \leq e).$$

Hence $F : \mathcal{C}^{\text{op}} \rightarrow \mathbf{Set}$ is a functor (a presheaf).

A.2.3 Sheaf Axioms

Locality and gluing are *axioms* that distinguish sheaves among presheaves; for an arbitrary presheaf on an arbitrary space, both fail in general. We record their statements here in the specific setting this paper uses — a presheaf induced by a functor on the finite context poset, over the Alexandrov topology — where, as discussed in Section 6.4, every open set is a union of minimal open neighbourhoods and both conditions are satisfied automatically. They are stated for completeness, not as a technical contribution.

Theorem A.3 (Locality, Alexandrov setting). Let F be the sheaf on $(X, \mathcal{T}_A)$ induced by a functor on the context poset, $U \in \mathcal{T}_A$, $\{U_i\}$ an open cover of U , and $s, t \in F(U)$. If $\rho_{U, U_i}(s) = \rho_{U, U_i}(t)$ for all i , then $s = t$.

Proof Sketch. Every $x \in U$ lies in some U_i , and $\mathcal{B}(x) \subseteq U_i$, so s and t agree on every minimal neighbourhood contained in U ; sections of the induced sheaf are determined by their values on minimal neighbourhoods. $\square$

Theorem A.4 (Gluing, Alexandrov setting). Let F be as above, $U \in \mathcal{T}_A$, $\{U_i\}$ an open cover of U , and $\{s_i \in F(U_i)\}$ a compatible family: $\rho_{U_i, U_i \cap U_j}(s_i) = \rho_{U_j, U_i \cap U_j}(s_j)$ for all i, j . Then there exists a unique $s \in F(U)$ with $\rho_{U, U_i}(s) = s_i$ for all i .

Proof Sketch. A section of the induced sheaf over U is an assignment of a value to each minimal neighbourhood $\mathcal{B}(x) \subseteq U$, compatible under restriction. For each $x \in U$, pick i with $x \in U_i$ and take the value s_i assigns to $\mathcal{B}(x)$; compatibility ensures this is well-defined on overlaps. Uniqueness follows from Theorem A.3. $\square$

A.2.4 Canonical Equivalence

Theorem A.5 (Invertibility of the Canonical Mapping). Let $\phi : \mathcal{F} \rightarrow \mathfrak{F}$ be the canonical mapping from facts to canonical facts, and $\psi : \mathfrak{F} \rightarrow \mathcal{F}$ the inverse mapping. Then:

$$\psi(\phi(f)) = f, \quad \phi(\psi(\mathfrak{f})) = \mathfrak{f}$$

for all $f \in \mathcal{F}$, $\mathfrak{f} \in \mathfrak{F}$.

Cross-engine query equivalence between Σ_{KG} and Σ_{Sheaf} is stated and proved as Theorem 7.1 in Section 7, together with the canonical-form machinery (Definitions 7.1 and 7.2) it depends on; we do not restate it here to avoid a second, weaker copy of the same claim.

A.2.5 Structural Properties

Theorem A.6 (Open-Set Intersections). For any $c_1, c_2 \in \mathcal{C}$ in the Alexandrov topology $\mathcal{T}_A$:

$$U_{c_1} \cap U_{c_2} = \begin{cases} U_{c_1 \wedge c_2} & \text{if } c_1, c_2 \text{ are comparable (the meet is the more specific of the two),} \\ \emptyset & \text{otherwise.} \end{cases}$$

Proof. A fact identifier lies in $U_{c_1} \cap U_{c_2}$ iff its context d satisfies $d \leq c_1$ and $d \leq c_2$. If $c_1 \leq c_2$, the two conditions reduce to $d \leq c_1 = c_1 \wedge c_2$ (symmetrically for $c_2 \leq c_1$). If c_1 and c_2 are incomparable, then $d \leq c_1$ and $d \leq c_2$ would require both c_1 and c_2 to be prefixes of d , which is impossible: the two paths disagree at their first differing segment. Hence no such d exists and the intersection is empty. $\square$

Note that the longest common prefix of two incomparable contexts is their *join* $c_1 \vee c_2$ (most specific common ancestor), not their meet, and it bounds the union rather than the intersection: $U_{c_1} \cup U_{c_2} \subseteq U_{c_1 \vee c_2}$, with equality failing in general (the join's open set also contains facts from sibling branches).

Theorem A.7 (Preservation of Referential Integrity). For any knowledge state Σ and fact $f \in \Sigma$:

$$\text{subject}(f) \in \mathcal{E}_\Sigma, \quad \text{relation}(f) \in \mathcal{R}_\Sigma, \quad \forall \omega_j \in \vec{o} : \omega_j \in \mathcal{E}_\Sigma \text{ if reference.}$$

Theorem A.8 (Determinism of Query Execution). For a fixed query Q and fixed knowledge state Σ , the result $\llbracket Q \rrbracket_\Sigma$ is uniquely determined, independent of execution order, caching state, parallelism, or query history.

Theorem A.9 (Consistency of Global Sections). A family of local sections $\{s_i \in F(U_i)\}$ can be glued to a global section $s \in F(\mathcal{C})$ iff for every pair (i, j) :

$$\rho_{U_i, U_i \cap U_j}(s_i) = \rho_{U_j, U_i \cap U_j}(s_j).$$

A.3 Further Detail

Fully worked proofs of every theorem above, pseudocode for the algorithms described in Section 9, and the implementation-to-definition validation table are kept in the repository rather than reproduced here, since they change with the code and would go stale in a typeset PDF:

Proofs `research/proofs/`

Algorithms `research/mathematics/algorithms.md`

Validation table `research/mathematics/validation.md`

References

- [1] Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011.
- [2] Güneş Aluç, Olaf Hartig, M. Tamer Özsu, and Khuzaima Daudjee. Diversified stress testing of RDF data management systems. In *Proceedings of the 13th International Semantic Web Conference (ISWC)*, pages 197–212. Springer, 2014.
- [3] Renzo Angles. The property graph database model. *Proceedings of the Alberto Mendelzon International Workshop on Foundations of Data Management*, 1912:1–12, 2017.

- [4] Renzo Angles and Claudio Gutierrez. Survey of graph database models. *ACM Computing Surveys*, 40(1):1–39, 2008.
- [5] Christian Bizer and Andreas Schultz. The berlin SPARQL benchmark. *International Journal on Semantic Web and Information Systems*, 5(2):1–24, 2009.
- [6] Jeremy J. Carroll, Christian Bizer, Pat Hayes, and Patrick Stickler. Named graphs, provenance and trust. In *Proceedings of the 14th International Conference on World Wide Web (WWW)*, pages 613–622. ACM, 2005.
- [7] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.
- [8] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999.
- [9] Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. LUBM: A benchmark for OWL knowledge base systems. *Journal of Web Semantics*, 3(2–3):158–182, 2005.
- [10] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C recommendation, World Wide Web Consortium, 2013.
- [11] Olaf Hartig. Foundations of RDF* and SPARQL*: An alternative approach to statement-level metadata in RDF. In *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management and the Web (AMW)*, 2017.
- [12] Olaf Hartig, Pierre-Antoine Champin, Gregg Kellogg, and Andy Seaborne. RDF-star and SPARQL-star. W3C final community group report, World Wide Web Consortium, 2021.
- [13] Frank Manola and Eric Miller. RDF primer. W3C recommendation, World Wide Web Consortium, 2004.
- [14] Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. Don’t like RDF reification? making statements about statements using singleton property. In *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, pages 759–770. ACM, 2014.
- [15] Natasha Noy and Alan Rector. Defining n-ary relations on the semantic web. W3C working group note, World Wide Web Consortium, 2006.
- [16] RDFLib Team. rdflib: A python library for working with RDF, 2024.
- [17] Michael Robinson. Sheaves are the canonical data structure for sensor integration. *Information Fusion*, 36:208–224, 2017.
- [18] Gian-Carlo Rota. On the foundations of combinatorial theory I. theory of Möbius functions. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 2(4):340–368, 1964.
- [19] David I. Spivak. Functorial data migration. *Information and Computation*, 217:31–51, 2012.